



Rapport de stage

NLP - RAG SYSTEM

Réalisé par :

Rabab HANZAZ Yasmina EL HAFI Oumaima LOUALI

Encadré par:

Thierry Bertin Gardelle

Chaymae Benhammacht

Hamza Mouncif

Année universitaire 2024/2025

INTRODUCTION	5
Remerciements	6
Liste des figures et tableaux	7
Chapitre 1 – Fondements théoriques	8
I. Introduction	8
1. Contexte et motivation	8
2. Problématique	8
3. Objectifs du projet	8
4. Présentation des outils et technologies	9
II. État de l'art	10
1. La génération augmentée de récupération (RAG):	10
2. Le RAG agentique	13
3. Analyse comparative des cadres Agentic RAG	27
4. Modèles LLM	28
3. LangChain : structure et utilité dans les applications RAG	30
4. Limites des systèmes classiques de NLP (sans RAG)	33
Conclusion du chapitre	35
Chapitre 2 – Conception et Réalisation du Système	36
I. Choix du modèle	36
1. Critères de sélection	36
2. Choix final	37
II. Partie Backend (Serveur et logique RAG)	37
1. Architecture générale du backend	37
2. Technologies et bibliothèques utilisées	41
3. Endpoints API principaux	42
2. Pipeline du traitement du document	44
4. Fonctionnement des agents	60
Conclusion du processus implémentation	62
III. Partie Frontend (Interface utilisateur)	63
1. Architecture générale du frontend	63
2. Les composants de l'interface	66
3. Architecture détaillée des composants	72
4. Outils et technologies utilisés	84
Conclusion du chapitre	85
Chapitre 3 - Comparaison entre le système RAG proposé (RAGbot) et les autres projets existants (SciSpace Copilot)	86
I. Introduction à SciSpace Copilot	86
1. Fonctionnement technique (RAG simplifié)	86
2. Fonctionnalités principales	87
II. Comparaison	87
1. Architecture Générale	87
2. Fonctionnement RAG	88
3. Comparaison visuelle	89

Conclusion générale	92
Références et bibliographies	93

Rapport sur le stage effectué dans la Société :

3D SMART FACTORY à Mohammedia

Intitulé du stage PFE - NLP systeme RAG



Rabab HANZAZ étudiante en

Master Big Data & Cloud Computing Année 2023/2025

A l'université Ibn Tofail , Faculté des sciences Kénitra



Yasmina EL HAFI étudiante en

Master Sciences des données et systèmes intelligents Année 2023/2025

A Faculté Pluridisciplinaire de Nador



Oumaima LOUALI étudiante en

Master Big Data & Cloud Computing Année 2023/2025

A l'université Ibn Tofail , Faculté des sciences Kénitra



INTRODUCTION

Au cours des dernières années, les avancées dans le domaine du traitement du langage naturel (NLP) ont conduit à l'élaboration de modèles complexes capables de saisir et de produire un texte comparable à celui écrit par un humain. Ces modèles, spécifiquement les larges modèles de langage (LLM), ont montré des compétences exceptionnelles dans plusieurs applications, y compris les chatbots, la création de contenu, entre autres. Néanmoins, bien qu'ils affichent des performances remarquables, ces modèles font face à d'importants défis, particulièrement en termes de précision et de pertinence des données qu'ils délivrent. Selon l'un des problèmes les plus pressants, appelé « hallucination », les modèles peuvent produire des réponses inexactes ou dépassées en raison des contraintes de leurs données d'apprentissage.

Pour répondre à ces enjeux, une stratégie innovante nommée Génération Augmentée par Récupération (RAG) a été développée. RAG fusionne les aptitudes génératives des LLM avec un processus de récupération d'informations, autorisant le modèle à se référer à des documents externes et à les exploiter en temps réel. Cette incorporation renforce l'aptitude du modèle à délivrer des réponses exactes et contextuellement appropriées en exploitant les données mises à jour d'une base de données structurée.

Le modèle RAG opère en premier lieu par l'identification des documents appropriés en réponse à la demande de l'utilisateur, ensuite en produisant des réponses fondées sur le contenu de ces documents. Cette approche non seulement accroît la précision des réponses, mais elle enrichit aussi l'expérience de conversation en assurant que le chatbot peut citer des informations à jour et précises.

Nous déployons un système de chatbot dans le cadre de ce projet, qui exploite le modèle Llama 3 à travers le framework Ollama, orchestré grâce à LangChain. Ce dispositif est prévu pour incorporer un moteur RAG, qui rehaussera notablement la pertinence des réponses du chatbot en utilisant une base de données vectorielle indexant des documents externes.

Ce projet utilise FastAPI pour la création d'API Web de haute performance, MongoDB pour une gestion flexible du stockage des données, et divers systèmes de stockage vectoriel comme FAISS et ChromaDB afin d'assurer une indexation et un accès efficace aux documents. En exploitant ces avancées technologiques, le but du projet est de concevoir un chatbot résistant et intelligent, en mesure de proposer des réponses exactes et contextualisées, ce qui améliorera l'interaction et la satisfaction des utilisateurs.

En résumé, l'intégration de RAG dans les systèmes de chatbot représente une avancée significative dans le domaine de la PNL, car elle permet de surmonter les limites des LLM traditionnels et d'ouvrir la voie à des agents conversationnels plus fiables et informatifs.

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet. Merci sincèrement pour le temps précieux que vous nous avez accordé, pour la confiance que vous nous avez témoignée tout au long de ce parcours, et pour les explications claires qui ont enrichi notre compréhension. Vos encouragements constants, votre patience face à nos questions, ainsi que votre écoute attentive ont été des sources essentielles de motivation et de progression. Grâce à votre soutien, ce travail a pu aboutir dans les meilleures conditions.

*Nos remerciements les plus chaleureux vont également à l'entreprise **3D SMART FACTORY** pour nous avoir offert l'opportunité d'effectuer ce stage de fin d'études dans un environnement aussi stimulant et enrichissant. Nous exprimons toute notre reconnaissance à **Monsieur Thierry Bertin Gardelle**, **Monsieur Hamza Mouncif** et **Madame Chaymae Benhammact** pour leur accueil, leur accompagnement attentif et la qualité de leurs enseignements. Leur confiance, leur disponibilité et leur professionnalisme ont grandement contribué à la réussite de cette expérience.*

Liste des figures et tableaux

Figure 1: architecture du RAG

Figure 2: le flux conceptuel de l'utilisation de RAG

Figure 3: présentation de la RAG agentique

Figure 4: Naïve RAG

Figure 5: RAG avancé

Figure 6: RAG modulaire

Figure 7: Illustration du flux de travail d'enchaînement d'invites

Figure 8: Flux de travail de routage d'illustrations

Figure 9: Illustration du flux de travail de parallélisation

Figure 10: Illustration du flux de travail Orchestrator-Workers

Figure 11: Illustration du flux de travail de l'évaluateur-optimiseur

Figure 12 : Aperçu des systèmes RAG multi-agents

Figure 13: architecture de LangChain

Tableau 1:Analyse comparative des paradigmes RAG

Tableau 2: Analyse comparative des cadres Agentiques RAG

Tableau 3 : Architecture Générale du système

Tableau 4: fonctionnement RAG

Chapitre 1 – Fondements théoriques

I. Introduction

1. Contexte et motivation

L'intelligence artificielle générative a connu un essor fulgurant ces dernières années, notamment avec l'émergence des modèles de langage à grande échelle (LLM) tels que GPT-4, Llama 3 et Mistral. Ces modèles ont démontré des capacités impressionnantes en matière de compréhension et de génération de texte, permettant le développement d'applications conversationnelles avancées. Toutefois, un défi majeur persiste : ces modèles sont limités par leurs données d'entraînement et peuvent parfois produire des réponses erronées ou obsolètes, un phénomène connu sous le nom de hallucination.

Pour pallier cette limitation, une approche appelée Retrieval-Augmented Generation (RAG) a été développée. Cette technique combine les capacités de génération d'un LLM avec un mécanisme de récupération d'information à partir d'une base documentaire. Ainsi, un chatbot basé sur RAG peut fournir des réponses plus précises et contextuelles en accédant à des sources d'informations mises à jour en temps réel.

Dans ce projet, nous mettons en œuvre un système de chatbot intelligent basé sur le modèle Llama 3 via Ollama et orchestré avec LangChain. Ce chatbot intégrera un moteur RAG afin d'améliorer la pertinence des réponses en exploitant des documents externes indexés dans une base de données vectorielle.

2. Problématique

Comment concevoir un système de conversation intelligent capable de fournir des réponses précises et contextualisées à partir d'un corpus documentaire spécifique, tout en tirant parti des avancées récentes en matière de modèles de langage ?

3. Objectifs du projet

L'objectif principal de ce projet est de concevoir et implémenter un chatbot RAG nous visons à :

- Développer un système de chat basé sur LangChain et Ollama.
- Intégrer une base documentaire pour améliorer la pertinence des réponses.
- Optimiser le système RAG pour garantir un équilibre entre temps de réponse et qualité de l'information.
- Évaluer les performances du chatbot, en le comparant à d'autres modèles.
- Développer une interface utilisateur complète à l'aide de Next.js, permettant :
 - D'envoyer un document PDF à analyser.
 - De poser des questions en langage naturel à partir du contenu du document fourni.
 - De consulter les réponses générées par le modèle de manière interactive (chatbot).
- Assurer une expérience utilisateur fluide et ergonomique, tout en maintenant la pertinence et la rapidité du système.

Ce projet s'inscrit dans un contexte où les applications conversationnelles sont de plus en plus utilisées dans divers domaines (assistance client, éducation, recherche d'information). L'ajout d'un module RAG permettrait d'améliorer significativement leur efficacité.

4. Présentation des outils et technologies

Pour réaliser ce projet, nous nous appuyons sur les technologies suivantes :

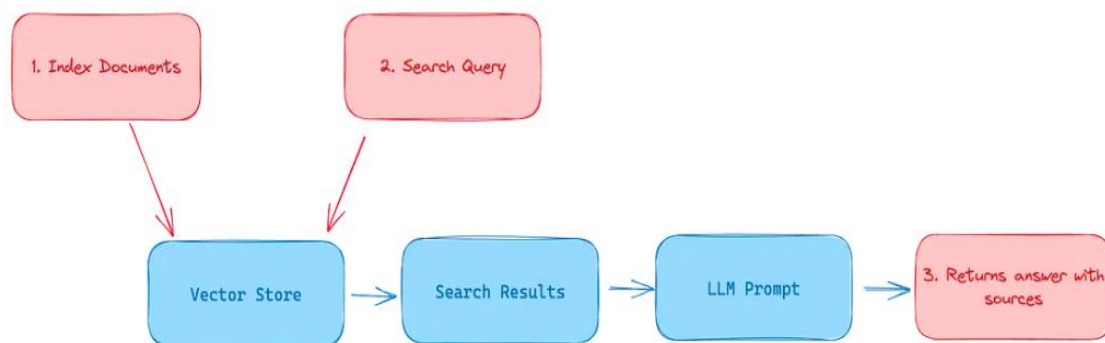
- Llama 3 : Un modèle de langage performant développé par Meta, utilisé via Ollama pour un déploiement local efficace.
- LangChain : Une bibliothèque permettant d'orchestrer les LLMs et d'intégrer des capacités de récupération d'information.
- Vector Store (Base vectorielle, ex : FAISS, ChromaDB, Pinecone) : Permet d'indexer et de rechercher efficacement des documents sous forme d'embeddings.
- FastApi: un framework Web hautes performances permettant de créer des API de services basées sur HTTP dans Python 3.8+
- Ollama : Un framework optimisé pour exécuter localement des modèles LLM de manière efficace.
- MongoDB : Base NoSQL orientée documents, parfaite pour la flexibilité des données non structurées issues des documents textuels.
- TypeScript : Typage statique pour une meilleure robustesse du code.
- Next.js : Framework React full-stack performant, idéal pour les applications web modernes et SSR (Server-Side Rendering).

Grâce à cette combinaison de technologies, notre chatbot pourra fournir des réponses contextuelles, précises et mises à jour, surpassant ainsi les modèles traditionnels qui reposent uniquement sur des connaissances statiques.

II. État de l'art

1. La génération augmentée de récupération (RAG):

L'optimisation du résultat d'un large modèle de langage (LLM) est le processus connu sous le nom de génération augmentée de récupération (RAG). Ainsi, elle s'appuie sur une base de connaissances externe fiable, distincte des sources de données qu'elle utilise pour sa formation, avant de produire une réponse. Les modèles de langage de grande envergure (LLM) sont formés à partir d'une vaste quantité de données et font appel à des milliards de paramètres pour produire des résultats inédits dans des domaines tels que la réponse à des interrogations, la traduction linguistique et le complétage de phrases. La RAG amplifie les compétences déjà considérables des LLM en les adaptant à des secteurs précis ou aux référentiels internes d'une organisation, tout cela sans l'obligation de reformation du modèle. Il s'agit d'une approche économique pour améliorer les résultats du LLM et qu'ils restent cohérents, précis et utiles dans de nombreux contextes.



L'architecture la plus primitive pour la génération augmentée par récupération.

Figure 1: architecture du RAG

Pour appréhender la génération augmentée par récupération, analysons-la selon ses deux éléments majeurs : les modèles de récupération et les modèles génératifs:

Modèles de recherche : Ces modèles sont élaborés pour récupérer des renseignements appropriés d'un corpus de documents ou d'une base de données. Ils ont tendance à recourir à des méthodes comme la recherche d'informations ou la recherche sémantique pour déterminer les données les plus appropriées en réponse à une requête spécifique. Les modèles axés sur la recherche sont idéaux pour dénicher des informations exactes et spécifiques, cependant, ils ne sont pas conçus pour produire du contenu inventif ou innovant.

Modèles génératifs : Pour leur part, les modèles génératifs sont élaborés pour créer du contenu inédit à partir d'une suggestion ou d'un contexte spécifique. Ces LLM

(maintenant détaillés sur toute la toile) exploitent une vaste base de données d'apprentissage pour assimiler les motifs et les structures du langage naturel. Les modèles génératifs ont la capacité de produire un texte inventif et logique, cependant ils peuvent rencontrer des problèmes pour assurer la véracité ou l'adéquation dans un contexte précis.

a. Fonctionnement de la génération augmentée de récupération

Sans RAG, le LLM puise dans les données fournies par l'utilisateur et génère une réponse en se basant sur les informations dont il a été formé ou celles déjà présentes dans sa base de connaissances. Avec RAG, un composant de recherche d'informations est intégré, exploitant les requêtes de l'utilisateur pour initialement dénicher des données issues d'une nouvelle source. La requête de l'utilisateur ainsi que les données pertinentes sont toutes deux acheminées vers le LLM. Le LLM exploite les informations récentes ainsi que ses données d'apprentissage pour fournir des réponses de qualité supérieure. Les parties qui suivent offrent un aperçu du processus.

Créer des données externes

Les données externes, mises à part les données LLM d'origine, peuvent provenir de différentes sources telles que des API, des bases de données ou des références documentaires. Ces données peuvent se présenter sous différentes formes telles que des fichiers, des enregistrements de base de données ou de longs textes. Une autre méthode d'IA, l'intégration de modèles linguistiques, transforme les données en représentations numériques et les enregistre dans une base de données vectorielle, constituant ainsi une bibliothèque de connaissances pour les modèles d'IA générative.

Récupérer les informations pertinentes

L'étape suivante consiste à effectuer une recherche de pertinence, à convertir les requêtes des utilisateurs en représentations vectorielles et à les associer à des bases de données vectorielles. Un chatbot intelligent peut répondre à des questions RH, telles que « Combien de jours de congés avez-vous ? ». Le système récupère les documents pertinents et les anecdotes des employés, calcule et établit la pertinence à l'aide de calculs et de représentations vectorielles mathématiques.

Augmenter l'invite LLM

Le modèle RAG améliore les instructions de l'utilisateur en intégrant des données pertinentes, en utilisant des techniques d'ingénierie rapide pour une communication efficace avec le LLM, permettant ainsi aux grands modèles linguistiques de produire des réponses précises aux demandes de l'utilisateur.

Mettre à jour les données externes

Le modèle RAG améliore les instructions de l'utilisateur en intégrant des données pertinentes, en utilisant des techniques d'ingénierie rapide pour une communication efficace avec le LLM, permettant ainsi aux grands modèles linguistiques de produire des réponses précises aux demandes de l'utilisateur.

Le schéma suivant montre le flux conceptuel de l'utilisation de RAG avec des LLM.

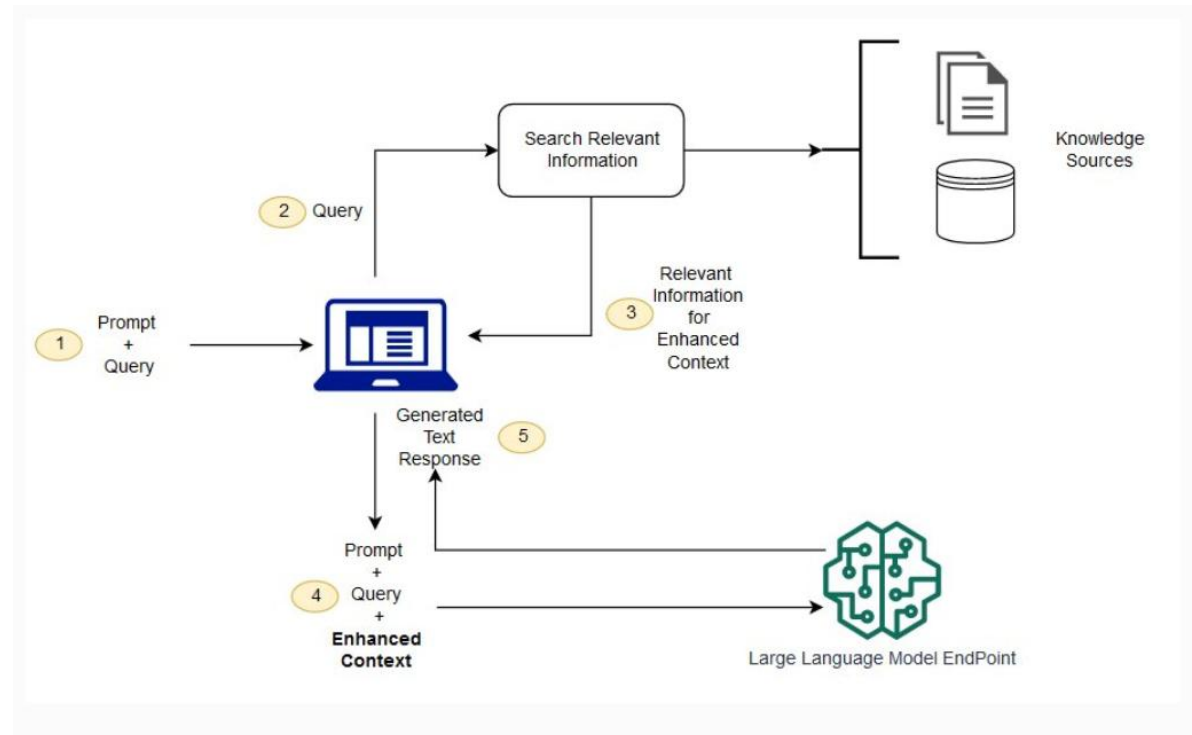


Figure 2: le flux conceptuel de l'utilisation de RAG

b. Avantages du RAG

La technologie RAG apporte plusieurs avantages aux efforts d'IA générative d'une organisation.

Mise en œuvre rentable:

Le développement d'un chatbot commence généralement avec un modèle de base, les systèmes d'IA générale (FM) étant des LLM accessibles par API. Toutefois, le coût financier et informatique élevé de la reconversion FM rend l'approche RAG plus lucrative pour l'introduction de nouvelles données dans les LLM.

Informations actuelles:

Bien que les sources de données soient appropriées pour un LLM, il peut être difficile de maintenir leur pertinence. Le RAG offre aux développeurs la possibilité de fournir des recherches récentes, des statistiques ou des actualités en lien avec les modèles générationnels, en établissant un lien direct entre le LLM et les médias sociaux, les sites d'information ou d'autres sources d'informations actuelles, puis de transmettre

ces informations à jour aux utilisateurs.

Confiance accrue des utilisateurs:

Grâce à la RAG, le LLM est capable de fournir des informations précises avec attribution de source, y compris des citations et des références. De plus, les utilisateurs ont la possibilité de consulter les documents sources pour obtenir davantage de détails et d'exactitude.

Plus de contrôle pour les développeurs:

RAG permet aux développeurs d'améliorer et de tester plus efficacement les applications de chat, de gérer et d'ajuster les sources d'informations LLM, de restreindre l'accès à des informations sensibles selon divers niveaux d'autorisation, d'assurer des réponses pertinentes et de résoudre les problèmes lorsque le LLM se réfère à des sources inexacts pour certaines questions. Cela permet aux entreprises d'exploiter la technologie de l'IA générative avec plus de confiance pour une diversité d'applications plus large.

2. Le RAG agentique

2.1 Présentation de la génération augmentée par récupération (RAG)

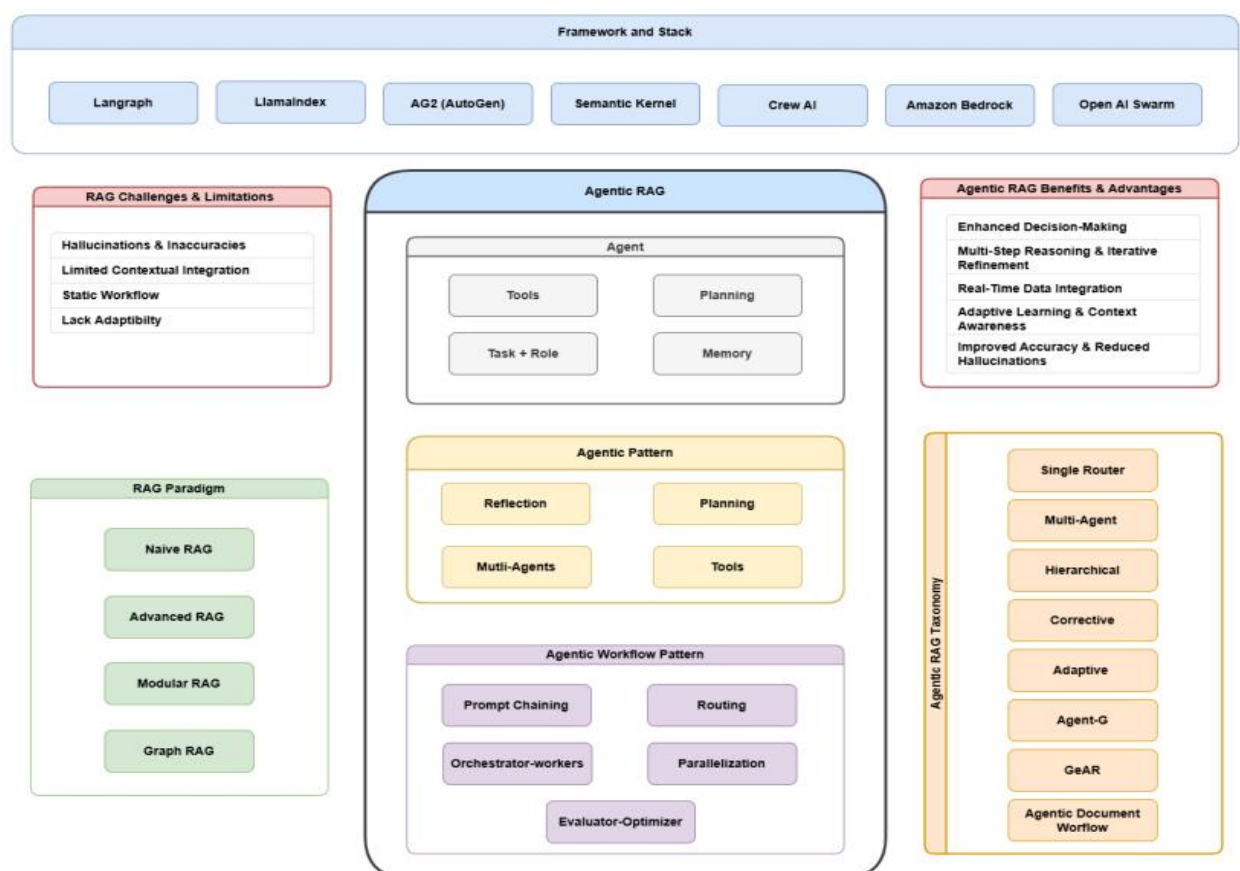


Figure 3: présentation de la RAG agentique

La génération augmentée par la récupération (RAG) constitue une avancée majeure dans le domaine de l'intelligence artificielle, fusionnant les aptitudes génératives des Grands Modèles Linguistiques (LLMs) avec l'extraction de données en temps réel. Bien que les LLMs aient montré des compétences exceptionnelles en traitement du langage naturel, leur dépendance à des données pré-entraînées statiques conduit souvent à des réponses périmées ou incomplètes. RAG surmonte cette contrainte en extrayant de manière dynamique des informations pertinentes provenant de sources externes et en les intégrant au processus de génération, permettant ainsi d'obtenir des résultats contextuellement précis et actualisés.

2.2 Évolution des paradigmes RAG

Le domaine de la génération augmentée par récupération (RAG) a connu une évolution notable pour répondre à la complexité croissante des applications du monde réel, où l'exactitude contextuelle, l'évolutivité et le raisonnement par étapes sont essentiels. Ce qui a commencé comme une simple recherche basée sur des mots-clés s'est transformé en systèmes sophistiqués, modulaires et adaptatifs, capables d'intégrer diverses sources de données et de processus décisionnels autonomes. Cette progression met en évidence la demande croissante de systèmes RAG capables de gérer efficacement et avec compétence des requêtes complexes.

Cette partie analyse l'évolution des paradigmes RAG, mettant en avant les étapes clés de développement : RAG Naïf, RAG Avancé, RAG Modulaire, RAG Graphique et RAG Agentique, ainsi que leurs caractéristiques distinctives, atouts et contraintes. En comprenant l'évolution de ces paradigmes, les lecteurs peuvent reconnaître les progrès réalisés dans les capacités de récupération et de génération et leur mise en œuvre dans divers domaines.

a. RAG Naïve

Naïve RAG est un système de récupération simple basé sur des mots-clés qui utilise des ensembles de données statiques pour renforcer les capacités génératives d'un modèle de langue. C'est adapté aux requêtes basées sur les faits avec une complexité minimale, mais présente des limitations telles que **l'absence de prise en compte du contexte, des réponses fragmentées et des problèmes de montée en charge**. Malgré ces problèmes, les systèmes RAG naïfs constituent une base pour des paradigmes de génération augmentée par la récupération plus sophistiqués.

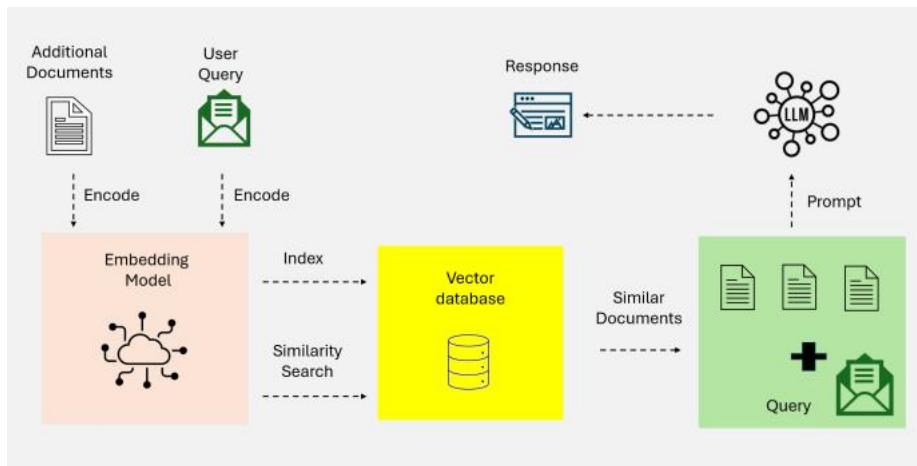


Figure 4: Naive RAG

b. RAG avancé

Les systèmes RAG avancés perfectionnent le RAG Naïve en intégrant une compréhension sémantique et des techniques de récupération améliorées. Parmi les fonctionnalités clés, on compte la recherche de vecteurs denses, le re-classage contextuel et la récupération itérative. Ces progrès rendent le RAG avancé idéal pour des applications de haute précision telles que la synthèse de recherche et les recommandations personnalisées, mais il est toujours confronté à des défis tels que la surcharge computationnelle et l'évolutivité limitée.

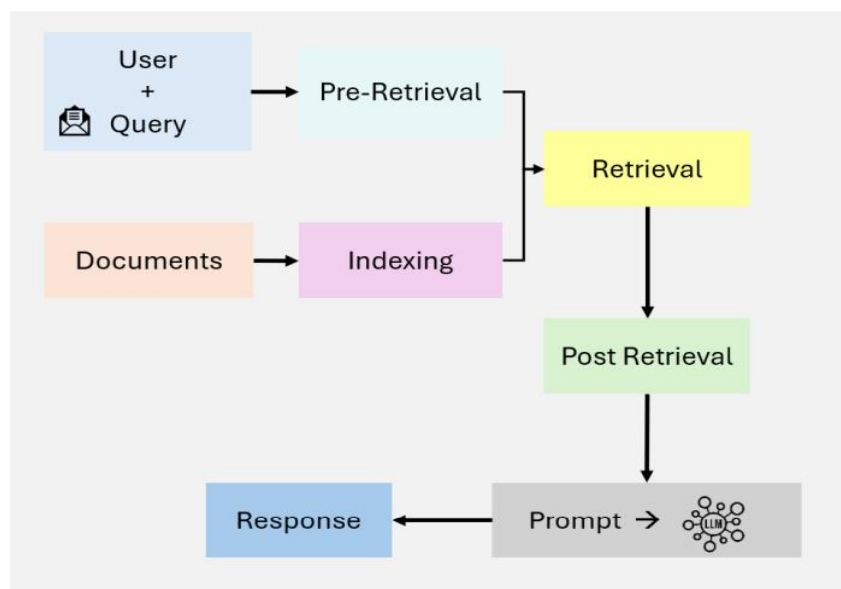


Figure 5: RAG avancé

c. RAG modulaire

Les systèmes RAG avancés perfectionnent le RAG Naïve en intégrant une

compréhension sémantique et des techniques de récupération améliorées. Parmi les fonctionnalités clés, on compte la recherche de vecteurs denses, le re-classement contextuel et la récupération itérative. Ces progrès rendent le RAG avancé idéal pour des applications de haute précision telles que la synthèse de recherche et les recommandations personnalisées, mais il est toujours confronté à des défis tels que la surcharge computationnelle et l'évolutivité limitée.

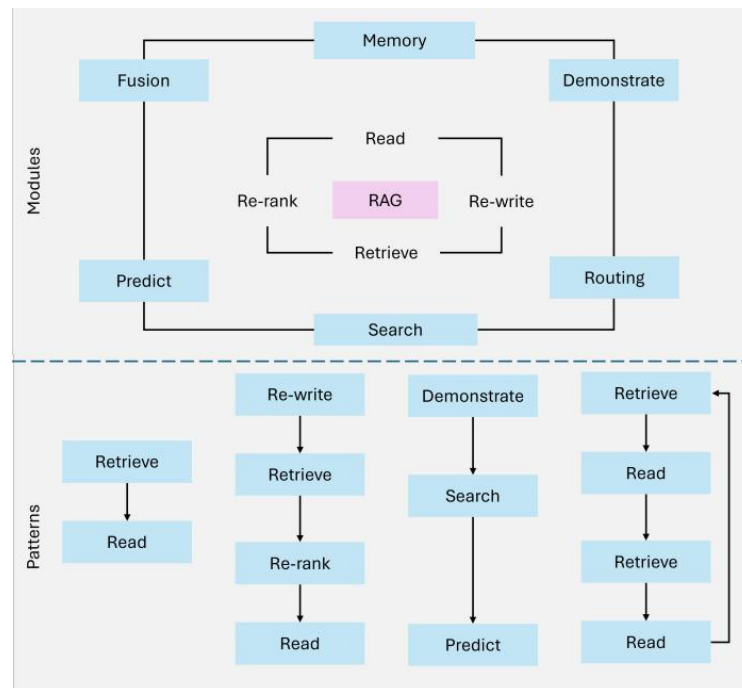


Figure 6: RAG modulaire

d. RAG graphique

Graph RAG est un système qui incorpore des structures de données basées sur le graphisme pour améliorer le raisonnement à plusieurs sauts et l'enrichissement contextuel. Cela permet de générer des résultats plus riches et précis, en particulier pour les tâches nécessitant une compréhension relationnelle. Cependant, il présente des contraintes comme une capacité de mise à l'échelle restreinte, une dépendance aux données et la complexité de l'intégration. Malgré cela, il est adapté à des applications telles que le diagnostic en santé et la recherche juridique où le raisonnement sur les relations structurées est essentiel.

e. RAG agentique

L'Agentic RAG représente une rupture de paradigme en introduisant des agents autonomes pour la prise de décisions dynamiques et l'optimisation des flux de travail. Elle utilise des stratégies de raffinement itératif et de récupération adaptative pour traiter des requêtes complexes. Parmi les caractéristiques principales, on retrouve la prise de décision autonome, l'affinement itératif et l'optimisation du flux de travail.

Malgré des obstacles tels que la complexité de coordination, la surcharge computationnelle et les limitations d'échelle, Agentic RAG se distingue dans des domaines tels que le support client, l'analyse financière et les plateformes d'apprentissage adaptatif.

Tableau comparatif

Paradigme	Caractéristiques clés	Forces
RAG Naïf	Recherche basée sur les mots-clés (ex. : TF-IDF, BM25)	- Simple et facile à implémenter - Adapté aux requêtes factuelles
RAG Avancé	- Modèles de recherche dense (ex. : DPR) - Classement neuronal et réévaluation - Recherche multi-sauts	- Recherche de haute précision - Meilleure pertinence contextuelle
RAG Modulaire	- Recherche hybride (sparse et dense) - Intégration d'outils et d'API - Pipelines composables et spécifiques à un domaine	- Grande flexibilité et personnalisation - Adapté à des applications diverses - Scalabilité
RAG Graphique	- Intégration de structures basées sur des graphes - Raisonnement multi-sauts - Enrichissement contextuel via les nœuds	- Capacités de raisonnement relationnel - Réduction des hallucinations - Idéal pour les tâches sur données structurées
RAG Agentique	- Agents autonomes - Prise de décision dynamique - Raffinement itératif et optimisation des flux de travail	- Adaptable aux changements en temps réel - Scalable pour les tâches multi-domaines - Haute précision

Tableau 1:Analyse comparative des paradigmes RAG

2.3 Modèles de flux de travail agentiques : stratégies adaptatives pour une collaboration dynamique

a. Chaînage des invites : améliorer la précision grâce au traitement séquentiel

L'enchaînement de prompts est une méthode structurée qui décompose les tâches

complexes en plusieurs étapes, ce qui améliore la précision en simplifiant chaque sous-tâche. Il est particulièrement efficace lorsque les tâches peuvent être divisées en sous-tâches spécifiques, et s'avère utile pour des missions telles que la traduction de contenu marketing ou la rédaction de documents.

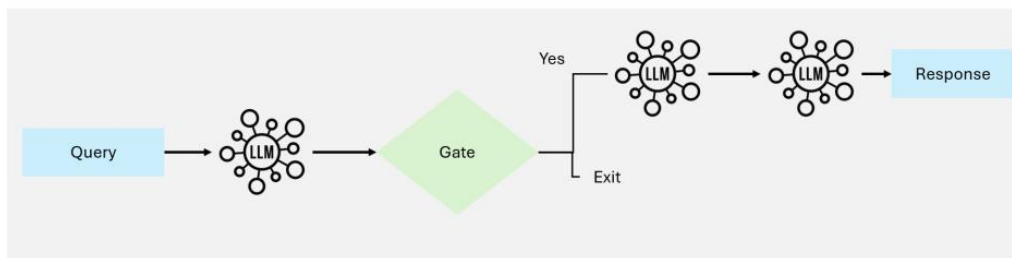


Figure 7: Illustration du flux de travail d'enchaînement d'invites

b. Routage : diriger les entrées vers des processus spécialisés

Le routage classe les entrées et les achemine vers des processus spécialisés, améliorant ainsi l'efficacité et la qualité de la réponse. C'est parfait pour gérer différents types d'entrées, comme les questions du service client, et pour attribuer les requêtes simples à de plus petits modèles afin d'optimiser les coûts.

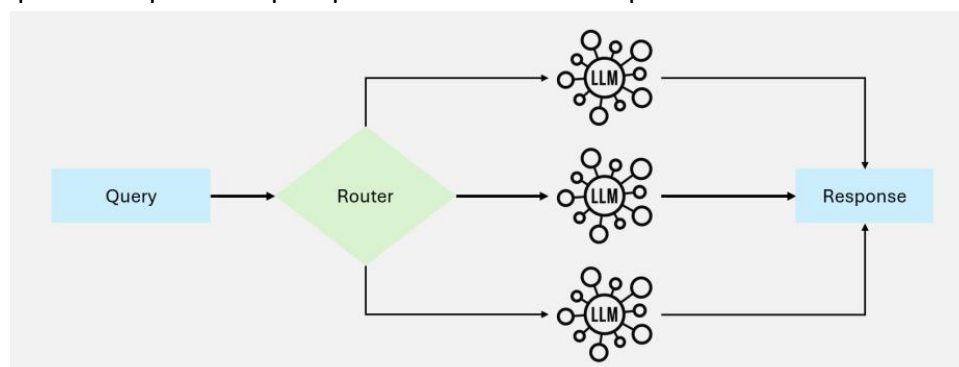


Figure 8: Flux de travail de routage d'illustrations

c. Parallélisation : accélération du traitement grâce à l'exécution simultanée

La parallélisation segmente les tâches en processus autonomes, diminuant ainsi la latence et optimisant le rendement. On peut le classer en deux catégories : sectionnement et vote, utilisés pour des tâches telles que la modération de contenu et la vérification croisée du code à la recherche de vulnérabilités.

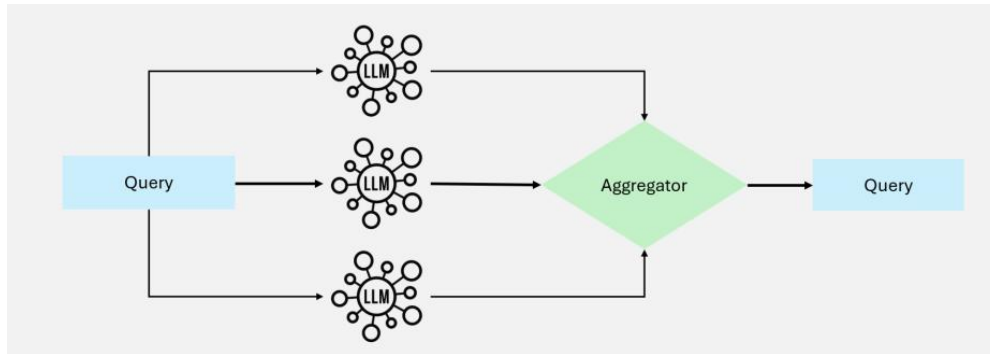


Figure 9: Illustration du flux de travail de parallélisation

d. Orchestrator-Workers : délégation dynamique des tâches

Le flux de travail utilise un modèle d'orchestrateur central pour diviser dynamiquement les tâches en sous-tâches, les attribuer à des modèles de travailleurs spécialisés et compiler les résultats, s'adaptant à la complexité variable des entrées, ce qui le rend idéal pour les tâches nécessitant une adaptation en temps réel.

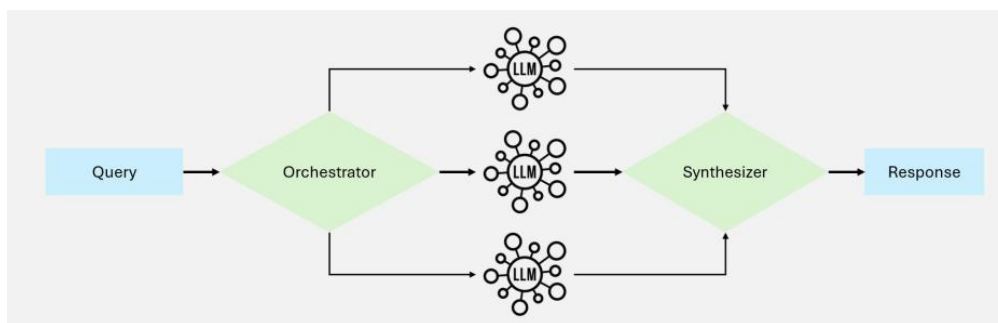


Figure 10: Illustration du flux de travail Orchestrator-Workers

e. Évaluateur-Optimiseur : Affiner les résultats par itération

Le processus itératif d'évaluation et d'optimisation améliore constamment le contenu en produisant une première version qui est ensuite perfectionnée en fonction des retours, surtout lorsqu'il existe des critères d'évaluation précis, pour les traductions littéraires et les requêtes de recherche à plusieurs tours.

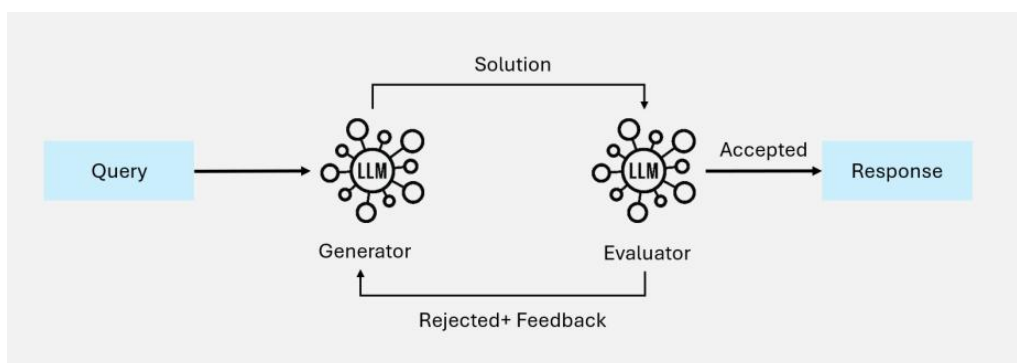


Figure 11: Illustration du flux de travail de l'évaluateur-optimiseur

2.4 Taxonomie des systèmes RAG agentiques

Les systèmes de génération augmentée par récupération agentique (RAG) peuvent être classés en différentes structures architecturales selon leur complexité et leurs principes de conception. Cela englobe les architectures à agent unique, les systèmes multi-agents et les architectures agentiques hiérarchiques. Chaque cadre est conçu sur mesure pour répondre à des défis spécifiques et optimiser les performances pour diverses applications. Cette section présente une taxonomie détaillée de ces architectures, mettant en évidence leurs caractéristiques, avantages et contraintes.

a. Agent RAG à agent unique : routeur

Un RAG agentique mono-agent [30] : sert de système décisionnel centralisé où un agent unique gère la récupération, le routage et l'intégration des informations. Cette architecture simplifie le système en consolidant ces tâches au sein d'un agent unifié, ce qui le rend particulièrement efficace pour les configurations avec un nombre limité d'outils ou de sources de données.

Flux de travail

1. Soumission et évaluation des requêtes : Le processus commence lorsqu'un utilisateur soumet une requête. Un agent coordinateur (ou agent principal de récupération) reçoit la requête et l'analyse afin de déterminer les sources d'information les plus appropriées.

2. Sélection des sources de connaissances : En fonction du type de requête, l'agent coordinateur choisit parmi diverses options de récupération :

- Bases de données structurées : Pour les requêtes nécessitant un accès aux données tabulaires, le système peut utiliser un moteur de conversion de texte en SQL qui interagit avec des bases de données telles que PostgreSQL ou MySQL.
- Recherche sémantique : Lorsqu'il traite des informations non structurées, le système récupère les documents pertinents (par exemple, des PDF, des livres, des documents organisationnels) grâce à une recherche vectorielle.
- Recherche Web : Pour des informations contextuelles en temps réel ou plus larges, le système utilise un outil de recherche Web pour accéder aux données en ligne les plus récentes.
- Systèmes de recommandation : Pour des requêtes personnalisées ou contextuelles, le système exploite des moteurs de recommandation qui fournissent des suggestions sur mesure.

3. Intégration des données et synthèse LLM : Une fois les données pertinentes extraites des sources sélectionnées, elles sont transmises à un modèle de langage étendu (LLM). Ce modèle synthétise les informations collectées, intégrant des informations provenant de sources multiples pour obtenir une réponse cohérente et contextuellement pertinente.

4. Génération de résultats : Enfin, le système fournit une réponse complète et accessible à l'utilisateur qui répond à la requête initiale. Cette réponse est présentée

dans un format concis et exploitable, et peut éventuellement inclure des références ou des citations des sources utilisées.

Caractéristiques clés et avantages.

Simplicité Centralisée : Un agent unique gère toutes les tâches de récupération et d'acheminement, rendant l'architecture facile à concevoir, à mettre en œuvre et à entretenir.

- **Performance et Optimisation des Ressources :** Grâce à un nombre réduit d'agents et une coordination simplifiée, le système requiert moins de ressources informatiques et est capable de traiter les requêtes plus rapidement.
- **Routage Dynamique :** L'agent examine chaque requête en temps réel, choisissant la source de connaissance la plus pertinente (par exemple, base de données structurée, recherche sémantique, recherche web).
- **Polyvalence sur les Outils :** Prend en charge diverses sources de données et API externes, permettant des flux de travail structurés et non structurés.
- **Idéal pour les systèmes simples :** Adapté aux applications disposant de tâches clairement définies ou ayant des besoins d'intégration restreints (par exemple, la récupération de documents, les flux de travail basés sur SQL).

b. Systèmes RAG multi-agents :

Le RAG multi-agents représente une évolution modulaire et évolutive des architectures mono-agent, conçue pour gérer des flux de travail complexes et divers types de requêtes en exploitant plusieurs agents spécialisés. Au lieu de s'appuyer sur un seul agent pour gérer toutes les tâches (raisonnement, récupération et génération de réponses), ce système répartit les responsabilités entre plusieurs agents, chacun optimisé pour un rôle ou une source de données spécifique.

Flux de travail:

1. Soumission de requête : Le processus commence par une requête utilisateur, reçue par un agent coordinateur ou un agent de récupération principal. Cet agent agit comme orchestrateur central, déléguant la requête à des agents de récupération spécialisés en fonction de ses besoins.

2. Agents de récupération spécialisés : La requête est répartie entre plusieurs agents de récupération, chacun se concentrant sur un type spécifique de source de données ou de tâche. Exemples :

- Agent 1 : Gère les requêtes structurées, telles que l'interaction avec des bases de données SQL telles que PostgreSQL ou MySQL.
- Agent 2 : Gère les recherches sémantiques pour récupérer des données non structurées à partir de sources telles que des PDF, des livres ou des documents internes.
- Agent 3 : Se concentre sur la récupération d'informations publiques en temps réel à partir de recherches web ou d'API.
- Agent 4 : Spécialisé dans les systèmes de recommandation, il fournit des

suggestions contextuelles basées sur le comportement ou le profil des utilisateurs.

3. Accès aux outils et récupération des données : Chaque agent achemine la requête vers les outils ou sources de données appropriés au sein de son domaine, tels que :

- Recherche vectorielle : Pour la pertinence sémantique.
- Conversion de texte en SQL : Pour les données structurées.
- Recherche web : Pour les informations publiques en temps réel.
- API : Pour l'accès à des services externes ou à des systèmes propriétaires.

Le processus de récupération est exécuté en parallèle, ce qui permet un traitement efficace de divers types de requêtes.

4. Intégration des données et synthèse LLM : Une fois la récupération terminée, les données de tous les agents sont transmises à un modèle de langage étendu (LLM). Ce modèle synthétise les informations récupérées en une réponse cohérente et contextuellement pertinente, intégrant de manière transparente les informations provenant de sources multiples.

5. Génération de résultats : Le système génère une réponse complète, qui est renvoyée à l'utilisateur dans un format concis et exploitable.

Principales fonctionnalités et avantages

- Modularité : Chaque agent fonctionne indépendamment, permettant l'ajout ou la suppression d'agents en toute transparence, en fonction des exigences du système.
- Évolutivité : Le traitement parallèle par plusieurs agents permet au système de gérer efficacement des volumes de requêtes importants.
- Spécialisation des tâches : Chaque agent est optimisé pour un type de requête ou de source de données spécifique, améliorant ainsi la précision et la pertinence de la récupération.
- Efficacité : En répartissant les tâches entre des agents spécialisés, le système minimise les goulots d'étranglement et améliore les performances des flux de travail complexes.
- Polyvalence : Convient aux applications couvrant plusieurs domaines, notamment la recherche, l'analyse, la prise de décision et le support client.

Défis

- Complexité de la coordination : La gestion de la communication inter-agents et de la délégation des tâches nécessite des mécanismes d'orchestration sophistiqués.
- Surcharge de calcul : Le traitement parallèle de plusieurs agents peut augmenter l'utilisation des ressources.
- Intégration des données : La synthèse des résultats de diverses sources en une réponse cohérente est complexe et nécessite des capacités LLM avancées.

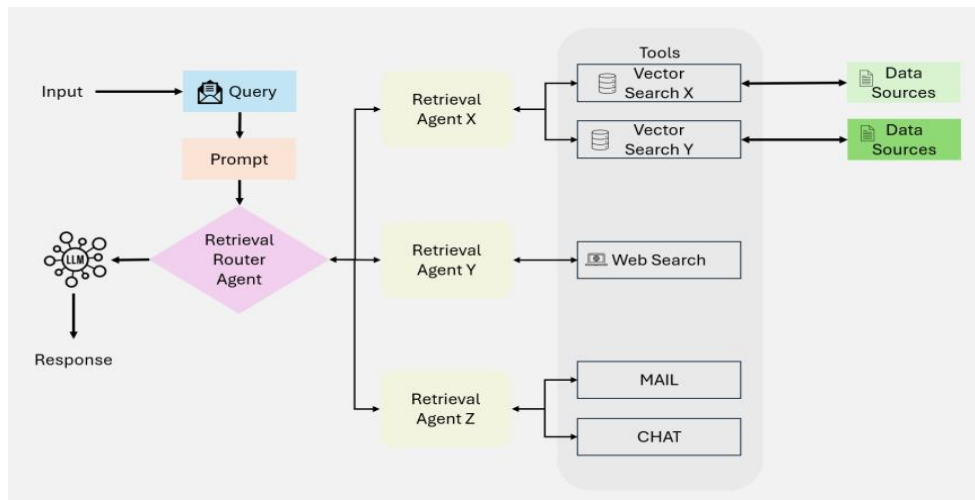


Figure 12 : Aperçu des systèmes RAG multi-agents

c. Systèmes RAG hiérarchiques et agentiques

Systèmes RAG hiérarchiques et agentiques : [14] les systèmes utilisent une approche structurée et multi-niveaux pour la recherche et le traitement de l'information, améliorant ainsi l'efficacité et la prise de décision stratégique. Les agents sont organisés hiérarchiquement, les agents de niveau supérieur supervisant et dirigeant les agents de niveau inférieur. Cette structure permet une prise de décision multi-niveaux, garantissant que les requêtes sont traitées par les ressources les plus appropriées.

Flux de travail

- 1. Réception de la requête** : Un utilisateur soumet une requête, qui est reçue par un agent de niveau supérieur chargé de l'évaluation initiale et de la délégation.
- 2. Prise de décision stratégique** : L'agent de niveau supérieur évalue la complexité de la requête et décide des agents subordonnés ou des sources de données à prioriser. Certaines bases de données, API ou outils de recherche peuvent être jugés plus fiables ou pertinents selon le domaine de la requête.
- 3. Délégation aux agents subordonnés** : L'agent de niveau supérieur attribue des tâches à des agents de niveau inférieur spécialisés dans des méthodes de recherche spécifiques (par exemple, bases de données SQL, recherche web ou systèmes propriétaires). Ces agents exécutent les tâches qui leur sont assignées de manière indépendante.
- 4. Agrégation et synthèse** : Les résultats des agents subordonnés sont collectés et intégrés par l'agent de niveau supérieur, qui synthétise les informations en une réponse cohérente.
- 5. Livraison de la réponse** : La réponse finale synthétisée est renvoyée à l'utilisateur, garantissant ainsi une réponse à la fois complète et contextuellement pertinente.

Principaux avantages et fonctionnalités :

- **Priorisation stratégique** : Les agents de niveau supérieur peuvent hiérarchiser les sources de données ou les tâches en fonction de la complexité,

de la fiabilité ou du contexte de la requête.

- Évolutivité : La répartition des tâches entre plusieurs niveaux d'agents permet de traiter des requêtes très complexes ou multidimensionnelles.
- Prise de décision optimisée : Les agents de niveau supérieur appliquent une supervision stratégique pour améliorer la précision et la cohérence globales des réponses.

Défis

- Complexité de la coordination : Maintenir une communication inter-agents robuste à plusieurs niveaux peut augmenter la charge d'orchestration.
- Allocation des ressources : Répartir efficacement les tâches entre les niveaux pour éviter les goulots d'étranglement n'est pas une mince affaire.

2.5 Correctif agentique RAG

RAG Correctif : introduit des mécanismes d'autocorrection des résultats de recherche, améliorant ainsi l'utilisation des documents et la qualité de la génération des réponses. En intégrant des agents intelligents au flux de travail, RAG Correctif assure un affinement itératif des documents contextuels et des réponses, minimisant ainsi les erreurs et maximisant la pertinence.

Idée clé du RAG Correctif : Son principe fondamental réside dans sa capacité à évaluer dynamiquement les documents récupérés, à effectuer des actions correctives et à affiner les requêtes afin d'améliorer la qualité des réponses générées.

Le RAG Correctif ajuste son approche comme suit :

- Évaluation de la pertinence des documents : La pertinence des documents récupérés est évaluée par l'agent d'évaluation de la pertinence. Les documents inférieurs au seuil de pertinence déclenchent des mesures correctives.
- Affinement et augmentation des requêtes : Les requêtes sont affinées par l'agent d'affinement des requêtes, qui s'appuie sur la compréhension sémantique pour optimiser la recherche et obtenir de meilleurs résultats.
- Récupération dynamique à partir de sources externes : Lorsque le contexte est insuffisant, l'agent de récupération de connaissances externes effectue des recherches sur le Web ou accède à des sources de données alternatives pour compléter les documents récupérés.
- Synthèse des réponses : Toutes les informations validées et affinées sont transmises à l'agent de synthèse des réponses pour la génération de la réponse finale.

Flux de travail :

Le système RAG correctif repose sur cinq agents clés :

- 1. Agent de récupération de contexte** : Chargé de récupérer les documents de contexte initiaux à partir d'une base de données vectorielle.
- 2. Agent d'évaluation de la pertinence** : Évalue la pertinence des documents récupérés et signale les documents non pertinents ou ambigus afin de mettre en œuvre des actions correctives.

3. Agent d'affinement des requêtes : Réécrit les requêtes pour améliorer la récupération, en s'appuyant sur la compréhension sémantique pour optimiser les résultats.

4. Agent de récupération de connaissances externes : Effectue des recherches sur le Web ou accède à des sources de données alternatives lorsque les documents de contexte sont insuffisants.

5. Agent de synthèse des réponses : Synthétise toutes les informations validées en une réponse cohérente et précise.

Principaux avantages et fonctionnalités :

- Correction itérative : Garantit une grande précision des réponses en identifiant et en corrigeant dynamiquement les résultats de recherche non pertinents ou ambigus.
- Adaptabilité dynamique : Intègre des recherches web en temps réel et l'affinement des requêtes pour une précision de recherche améliorée.
- Modularité des agents : Chaque agent effectue des tâches spécialisées, garantissant un fonctionnement efficace et évolutif.
- Assurance de la factualité : En validant l'ensemble du contenu récupéré et généré, le RAG correctif minimise le risque d'hallucinations ou de désinformation.

2.6 Adaptation agentique RAG

La génération augmentée par récupération adaptative (ADAG) améliore la flexibilité et l'efficacité des grands modèles de langage (LLM) en ajustant dynamiquement les stratégies de traitement des requêtes en fonction de la complexité de la requête entrante. Contrairement aux workflows de récupération statique, l'ADAG utilise un classificateur pour évaluer la complexité des requêtes et déterminer l'approche la plus appropriée, allant de la récupération en une seule étape au raisonnement en plusieurs étapes, voire en contournant complètement la récupération pour les requêtes simples.

Le RAG adaptatif est un système qui ajuste de manière dynamique les stratégies de récupération en fonction de la complexité d'une requête. Il est capable de traiter des requêtes simples, des requêtes basiques et des requêtes complexes. Le système repose sur trois éléments : un rôle de classificateur, la sélection dynamique de stratégie et l'intégration de LLM. Le classificateur évalue la complexité de la requête, tandis que le système s'appuie sur un modèle linguistique de taille réduite pour générer des réponses. Des interactions itératives entre le LLM et le classificateur permettent de perfectionner les requêtes complexes. Parmi les caractéristiques principales figurent l'adaptabilité dynamique, l'efficacité des ressources, la précision améliorée et la flexibilité. Ces caractéristiques peuvent être élargies pour inclure d'autres voies telles que des outils spécifiques à un domaine ou des API externes.

2.7 Graph-Based Agentic RAG

a. Agent-G : Cadre agentique pour les graphes RAG

Agent-G est un système de big data qui attribue de manière dynamique des missions de récupération à des agents spécialisés en utilisant des bases de connaissances graphiques et des documents textuels. Elle exploite les données structurées pour déterminer les rapports et les hiérarchies, tandis que les documents non structurés fournissent des informations contextuelles. Le système utilise un Module Critique pour juger la pertinence et la qualité des informations récupérées, ainsi que des Boucles de Rétroaction pour perfectionner le processus de récupération et de synthèse grâce à une validation itérative et à une nouvelle interrogation. Parmi ses caractéristiques principales, on compte un raisonnement approfondi, une adaptabilité dynamique, une précision améliorée et une modularité évolutive qui facilite l'ajout de nouveaux agents pour des missions spécialisées.

b. GeAR : Graph-Enhanced Agent for Retrieval-Augmented Generation (agent amélioré par les graphes pour la génération assistée par la recherche)

GeAR est un cadre agentique qui améliore les systèmes traditionnels de Génération Augmentée par Récupération (RAG) en intégrant des mécanismes de récupération basés sur des graphes. Il traite les difficultés rencontrées dans les scénarios de recherche multi-hop et améliore la capacité du système à gérer des requêtes complexes. Parmi les innovations majeures de GeAR figurent l'Expansion de Graph, qui élargit le processus de récupération pour intégrer des données structurées en graphes, et un Cadre d'Agent, qui intègre l'expansion de graphes afin de gérer les tâches de récupération plus efficacement. Le système fonctionne grâce à des éléments tels que le Module d'Expansion de Graphe, la Récupération Basée sur les Agents, et l'Intégration de LLM. Les caractéristiques clés comprennent une récupération multi-saut améliorée, une prise de décision autonome, une précision accrue et une capacité d'évolution. L'architecture modulaire du cadre agent facilite l'ajout de stratégies de récupération et de sources de données supplémentaires selon les nécessités.

2.8 Flux de documents Agentique dans Agentic RAG

Les flux de travail documentaires agentiques (ADW) représentent une approche novatrice de l'automatisation des travaux de connaissance qui fusionne la recherche, l'analyse et le raisonnement documentaire avec des agents intelligents. Ces flux de travail répondent aux limites du Traitement Intelligent des Documents (IDP) et du RAG en préservant l'état, en coordonnant les flux de travail en plusieurs étapes et en appliquant une logique spécifique au domaine sur les documents. Les systèmes ADW englobent l'analyse de documents et la structuration d'informations, le

maintien d'état à travers les processus, la récupération de connaissances, l'orchestration agentique et la production de résultats exploitables. Parmi les caractéristiques clés, on compte la gestion de l'état, l'orchestration en plusieurs étapes, l'intelligence spécifique au domaine, la capacité d'adaptation à grande échelle et l'amélioration de la productivité grâce à l'automatisation des tâches répétitives et à l'enrichissement de l'expertise humaine dans la prise de décisions.

3. Analyse comparative des cadres Agentic RAG

La table suivante offre une analyse comparative entre le RAG traditionnel, le RAG agentique et les flux de documents agentiques (ADW), en mettant en lumière sur leurs points forts, leurs faiblesses et les scénarios où chacun est le mieux adapté. Le RAG traditionnel propose une simplicité, le RAG Agentique améliore le raisonnement et l'évolutivité, tandis que l'ADW offre des flux de travail solides axés sur les documents pour l'automatisation et l'intégration.

Fonctionnalité	RAG Traditionnel	RAG Agentique	Flux de Travail Documentaires Agentiques (ADW)
Objectif	Tâches isolées de récupération et génération	Collaboration et raisonnement multi-agents	Flux de travail documentaires de bout en bout
Maintien du Contexte	Limité	Activé via des modules de mémoire	Maintient l'état à travers des flux de travail multi-étapes
Adaptabilité Dynamique	Minimale	Élevée	Adaptée aux flux de travail documentaires
Orchestration des Flux de Travail	Absente	Orchestration de tâches multi-agents	Intègre le traitement documentaire multi-étapes
Utilisation d'Outils/API Externes	Intégration basique (ex. : outils de récupération)	Étendue via des outils comme les API et bases de connaissances	Intègre profondément les règles métiers et outils spécifiques au domaine
Scalabilité	Limitée aux petits ensembles de données ou requêtes	Scalable pour les systèmes multi-agents	Scalabilité pour des flux de travail d'entreprise multi-domaines
Raisonnement	Basique (ex. :	Raisonnement	Raisonnement

Fonctionnalité	RAG Traditionnel	RAG Agentique	Flux de Travail Documentaires Agentiques (ADW)
Complexe	Q/R simple)	multi-étapes avec agents	structuré à travers des documents
Applications Principales	Systèmes de questions-réponses, récupération de connaissances	Raisonnement et connaissances multi-domaines	Révision de contrats, traitement de factures, analyse de réclamations Automatisation de
Forces	Simplicité, mise en place rapide	Haute précision, raisonnement collaboratif	bout en bout, intelligence spécifique au domaine
Défis	Mauvaise compréhension du contexte	Complexité de coordination	Surcharge de ressources, standardisation des domaines

Tableau 2: Analyse comparative des cadres Agentiques RAG

4. Modèles LLM

Les modèles de langage de grande taille (Large Language Models ou LLM) constituent un progrès significatif dans le secteur de l'intelligence artificielle et du traitement automatique du langage naturel (NLP). Basés sur des structures de réseaux neuronaux approfondis, en particulier les Transformers dévoilés par Vaswani et al. en 2017, ces modèles ont la capacité de produire, traduire, résumer ou répondre à des textes en langue naturelle avec aisance et cohérence.

a. Définition et principe de fonctionnement

Un LLM est un modèle qui a été formé sur un corpus de textes de très grande taille, parfois comportant plusieurs centaines de milliards de mots, dans le but d'assimiler les régularités statistiques du langage humain. Sa mission principale est de prévoir le mot suivant dans une suite donnée. En multipliant cette procédure à grande échelle, le modèle est capable de produire des phrases complètes, de répondre à des interrogations ou encore de synthétiser des documents.

Le modèle est formé de façon auto-supervisée, ce qui signifie qu'il se base sur les données elles-mêmes pour apprendre, sans avoir besoin d'annotations explicitement fournies par des humains. Des modèles à la pointe de la technologie, comme GPT-3, GPT-4 (OpenAI), PaLM (Google), Claude (Anthropic) ou LLaMA (Meta), possèdent des

dizaines, voire des centaines de milliards de paramètres.

Fonctionnement des LLM sur l'apprentissage automatique et apprentissage en profondeur

Les LLM s'appuient sur l'apprentissage automatique, une branche de l'intelligence artificielle, qui alimente un programme avec d'importantes quantités de données afin de repérer des caractéristiques sans intervention humaine. Les modèles d'apprentissage en profondeur (Deep Learning) sont capables de détecter des différences sans intervention humaine, même avec une contribution humaine minimale. Cette méthode utilise des probabilités pour apprendre, comme par exemple les caractères les plus fréquemment présents dans un texte français. Après avoir analysé des millions de phrases, il est capable d'apprendre à terminer ou à créer des phrases.

Réseaux neuronaux LLM

Le LLM s'appuie sur des réseaux neuronaux artificiels, qui sont des réseaux de neurones interconnectés par une série de « couches » : une entrée, une sortie et une ou plusieurs couches intermédiaires qui ne transmettent l'information que si leurs résultats dépassent un certain seuil.

Modèles transformateurs LLM

LLM utilise un type de réseau neuronal particulier appelé « transformateur » ou auto-attentif, capable d'apprendre le contexte, une caractéristique essentielle pour la langue humaine. Ces modèles exploitent une méthode mathématique nommée « auto-attention » pour identifier les relations subtiles entre les éléments de la séquence. Cette méthode permet au LLM de comprendre le langage humain, même lorsqu'il est flou ou mal défini. Elles intègrent également la sémantique, car elles associent les mots et les concepts en fonction de leur signification après les avoir vus ensemble des millions ou des milliards de fois.

b. Capacités des LLM

Les LLM disposent de capacités impressionnantes en :

- Compréhension du langage naturel (questions-réponses, inférence logique, classification, etc.)
- Génération de texte (complétion, rédaction, dialogue, etc.)

- Traduction automatique, résumé, réécriture et bien plus.
- Raisonnement implicite et traitement des connaissances encodées dans leurs poids.

Néanmoins, ils ne sont pas sans défauts : illusions (fabrication de faits), préjugés dus aux données d'apprentissage, grande consommation de ressources, et problèmes pour obtenir des informations récentes ou spécifiques à un secteur particulier.

c. Modèle utilisé dans ce projet : LLaMA 3.2:1B via Ollama

Pour ce projet, nous avons choisi **LLaMA 3.2:1B**, une version légère du modèle **LLaMA 3** (Large Language Model Meta AI), développé par **Meta AI**.

LLaMA représente une gamme de modèles open source conçus pour offrir des performances comparables à celles des modèles propriétaires, tout en restant accessibles aux chercheurs et développeurs.

La version **3.2**, publiée en **avril 2024**, apporte plusieurs améliorations par rapport aux versions précédentes :

- Meilleure compréhension des contextes longs.
- Performances accrues sur de nombreux benchmarks NLP.
- Optimisation de la vitesse d'inférence et de l'utilisation mémoire.

Le modèle **LLaMA 3.2:1B**, qui contient **environ 1 milliard de paramètres**, a été retenu pour sa légèreté et son efficacité, ce qui le rend parfaitement adapté à une exécution **locale via la plateforme Ollama**.

Ollama simplifie le déploiement et l'exécution de modèles de langage sur des ordinateurs personnels, sans nécessiter d'infrastructure cloud complexe. Cette approche garantit une meilleure confidentialité, une réduction des coûts et un meilleur contrôle du processus.

d. Rôle du LLM dans le système RAG

Dans une architecture **RAG (Retrieval-Augmented Generation)**, le modèle de langage joue un rôle central mais **complémentaire**. Il ne se limite plus à ses connaissances internes, mais exploite également **des informations extraites dynamiquement** à partir d'une base documentaire, afin de générer des réponses plus **pertinentes et actualisées**.

Dans ce projet, **LLaMA 3.2:1B** utilise les **segments textuels extraits de documents PDF, DOCX et TXT** lors de la phase de récupération, qu'il intègre ensuite à son **prompt** pour produire des réponses **contextualisées** destinées à l'utilisateur.

3. LangChain : structure et utilité dans les applications RAG

Dans le cadre des architectures RAG, la coordination entre les diverses phases – extraction de documents, élaboration du contexte, formulation de la réponse – peut vite s'avérer complexe. LangChain a été spécifiquement conçu pour répondre à cette exigence, en tant que cadre open-source qui facilite la construction, l'enchaînement et l'orchestration des éléments d'un système fondé sur les LLM.

Objectif de LangChain

LangChain a été développé pour faciliter l'intégration des modèles de langage dans des applications concrètes, qu'il s'agisse de moteurs de recherche sémantiques, d'assistants conversationnels, d'agents autonomes ou de systèmes RAG.

Son objectif est triple :

- Offrir une interface unifiée pour interagir avec des LLM, quels qu'ils soient (OpenAI, Hugging Face, Ollama, etc.).
- Faciliter la création de chaînes de traitement (chain-of-thought, RAG pipelines, agents, etc.).
- Gérer la connexion avec des sources de données externes (PDF, bases de données, APIs, vecteurs, etc.).

a. Composants principaux

LangChain repose sur plusieurs briques modulaires, notamment :

- PromptTemplates : gestion structurée des invites pour guider le comportement du LLM.
- Chains : assemblage d'étapes en pipeline (ex : retrieval -> context -> generation).
- Retrievers : modules de récupération d'information à partir d'un index vectoriel.
- Document Loaders : chargement et découpage de documents (PDF, txt, sites web...).
- Vector Stores : intégration avec des bases vectorielles (FAISS, Chroma, Pinecone...).
- Agents & Tools : logique plus complexe où le modèle peut appeler des outils externes (plugins, API, fonctions).

b. Utilité spécifique dans RAG

Dans une architecture RAG, LangChain agit comme chef d'orchestre entre les composants suivants :

- Chargement de documents (via un DocumentLoader).
- Découpage en chunks avec gestion des chevauchements.
- Indexation vectorielle avec embeddings (comme ceux de Hugging Face ou OpenAI).

- Recherche vectorielle à chaque requête utilisateur.
- Injection du contexte pertinent dans le prompt du LLM (template contrôlé).
- Génération finale via un modèle comme LLaMA 3.2 dans Ollama.

c. Fonctionnement du LangChain

Avec LangChain, les développeurs peuvent adapter un modèle de langage de manière flexible à des contextes métier spécifiques en désignant les étapes nécessaires pour produire le résultat souhaité.

Chaînes

Dans LangChain, le concept de chaînes est au cœur des différents éléments de l'IA, permettant ainsi de délivrer des réponses adaptées au contexte. Une chaîne est une succession d'actions automatisées allant de la demande de l'utilisateur jusqu'à la sortie du modèle. Par exemple, les développeurs ont la possibilité d'utiliser une chaîne pour :

- La connexion à différentes sources de données
- La génération de contenu unique
- La traduction de plusieurs langues
- La réponse aux questions des utilisateurs

Liens

Les chaînes sont faites de liens. Chaque action que les développeurs enchaînent pour former une séquence enchaînée est appelée lien. Grâce aux liens, les développeurs peuvent diviser les tâches complexes en plusieurs tâches plus petites. Voici des exemples de liens :

- Formatage des données saisies par l'utilisateur
- Envoi d'une requête à un LLM
- Extraction de données depuis le stockage dans le cloud
- Traduction d'une langue à l'autre

Dans le cadre LangChain, un lien accepte les entrées de l'utilisateur et les transmet aux bibliothèques LangChain pour traitement. LangChain permet également de réorganiser les liens pour créer différents flux de travail d'IA.

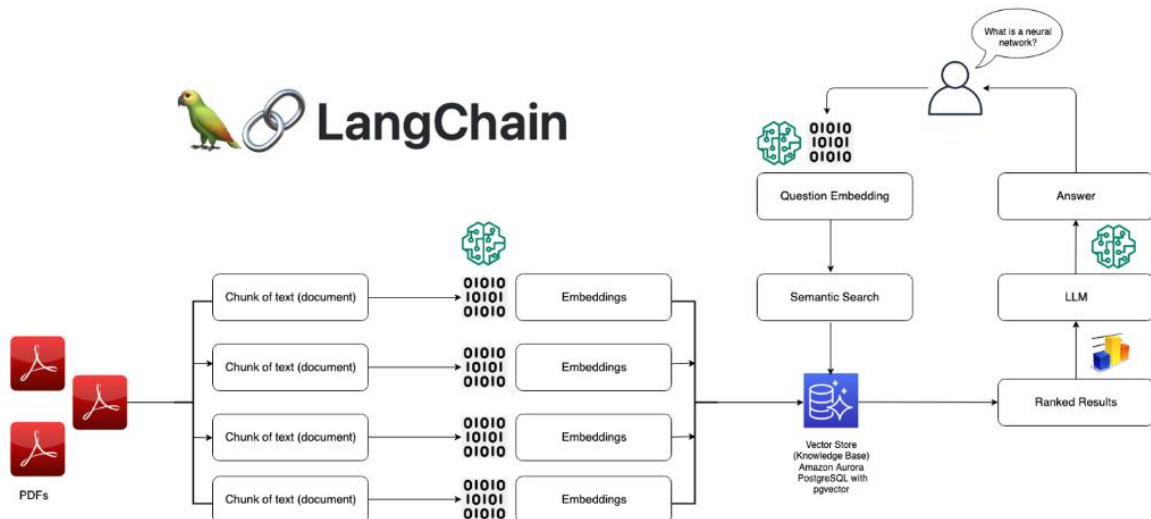


Figure 13: architecture de LangChain

Ainsi, LangChain permet de gérer automatiquement l'ensemble de la chaîne RAG, en limitant les erreurs manuelles et en rendant le système plus modulaire et maintenable. LangChain joue un rôle clé dans la gestion intelligente du flux de données entre l'utilisateur, les documents et le modèle LLM, permettant ainsi un chatbot RAG fonctionnel, robuste et évolutif.

4. Limites des systèmes classiques de NLP (sans RAG)

Les modèles linguistiques à grande échelle (LLM), tels que GPT, PaLM, Claude et LLaMA, ont prouvé leur aptitude exceptionnelle à produire des réponses naturelles, réaliser des opérations de raisonnement sophistiquées ou imiter des conversations. Cependant, lorsqu'ils sont employés sans un dispositif d'accès externe à l'information, ils possèdent plusieurs contraintes significatives qui limitent leur mise en application dans des systèmes spécialisés ou délicats.

a. Connaissance limitée au corpus d'entraînement

Les LLM sont entraînés sur des données fixes, et leur connaissance du monde s'arrête à la date de leur dernier entraînement. Cela signifie qu'ils ne peuvent pas :

- Accéder à de nouvelles informations (actualités, mises à jour juridiques, découvertes récentes, etc.).
- Répondre avec précision à des questions contextuelles ou spécifiques à un domaine (ex. : contenu d'un document interne, manuel technique, etc.).

b. Risque d' hallucinations

Sans référence externe fiable, les LLM peuvent générer des réponses qui paraissent plausibles mais qui sont en fait erronées, un phénomène connu sous le nom d'hallucination. C'est un souci majeur dans les secteurs sensibles (médical, juridique, scientifique), où l'exactitude est primordiale.

Exemple : un modèle peut inventer une citation, une source ou une procédure inexistante simplement pour compléter sa réponse.

c. Incapacité à raisonner sur des documents utilisateur

Les LLM classiques ne peuvent ni lire des documents PDF, ni accéder à des bases de données spécifiques, sauf si leur contenu a été intégré dans leur corpus d'entraînement. Cela les rend inutilisables dans des contextes où l'information est :

- Contextuelle ou confidentielle (ex. : entreprise, juridique, médical).
- Fourni dynamiquement par l'utilisateur (ex. : support client, e-learning).

d. Dépendance à la longueur du contexte

Même en tentant de fournir manuellement des données dans le prompt, les LLM sont contraints par la taille de la fenêtre de contexte (par exemple : 4K, 16K ou 32K tokens selon le modèle). Ce paramètre restreint le volume d'informations que le modèle peut exploiter à un instant précis, en particulier lorsqu'il s'agit de documents de grande taille.

Pourquoi RAG répond à ces limites ?

La structure RAG (Generation Augmentée par la Recherche) vise à remédier à ces points faibles en associant :

- Un moteur de recherche sémantique sur des documents externes ou spécifiques.
- Un modèle génératif qui intègre ces informations pour répondre de façon contextualisée, traçable et à jour.

Par conséquent, un système RAG ne se concentre plus uniquement sur ce que « le modèle sait », mais examine également ce que révèlent les données, un aspect crucial pour une utilisation fiable dans les domaines de l'entreprise, de l'éducation, de la santé ou du droit.

e. Projets existants

1. SciSpace Copilot

Type : Assistant de lecture de PDF scientifiques

Description : SciSpace Copilot permet à l'utilisateur de poser des questions sur un article scientifique (PDF). Il utilise un pipeline RAG pour extraire du

texte, l'indexer via un moteur de recherche sémantique (ex: FAISS), et générer des réponses avec un LLM comme GPT.

Tech utilisé : LangChain, FAISS, OpenAI GPT, PDF parsing (PyMuPDF), embedding (OpenAI, HuggingFace)

2.ChatPDF

Type : Chatbot interactif avec des fichiers PDF

Description : ChatPDF permet à l'utilisateur de discuter avec des documents PDF (rapports, thèses, manuels). Le système RAG extrait les textes, les vectorise, puis alimente un LLM pour répondre aux questions de l'utilisateur.

Tech utilisé : SentenceTransformers, FAISS, GPT-3.5, LangChain

3.AskYourPDF (plugin ou web)

Type : Plugin ou webapp pour interagir avec des PDF

Description : Similaire à ChatPDF, il permet d'uploader un document et d'en discuter. Le backend utilise l'approche RAG pour extraire du contexte avant de générer la réponse.

Tech utilisé : Gradio, Groq AI, Google Generative AI Embeddings, FAISS, GPT-3.5, LangChain

4. CaseLaw Chat (Harvard LIL)

Type : Chatbot pour documents juridiques

Description : Développé par la Harvard Library Innovation Lab, ce chatbot permet d'interagir avec des cas juridiques américains. Il utilise RAG pour relier la jurisprudence à des réponses générées par un modèle LLM.

Tech utilisé : Haystack, Elasticsearch, OpenAI, LangChain

Conclusion du chapitre

Dans ce premier chapitre, nous avons exploré les fondements théoriques qui donnent du sens à notre projet. On a vu que les modèles de langage comme LLaMA 3.2 sont incroyablement puissants, mais qu'ils ne savent pas tout – surtout quand il s'agit d'informations récentes ou très spécifiques. C'est là que la RAG entre en jeu : en ajoutant une couche de recherche intelligente, elle permet aux modèles de rester pertinents, même face à des questions complexes. Grâce à des outils comme LangChain et Ollama, notre chatbot pourra vraiment "comprendre" ce qu'on lui demande, en s'appuyant sur des données à jour, et pas juste sur ce qu'il a appris pendant son entraînement. Ce chapitre nous a permis de poser les bases. Dans le suivant, on passe à la pratique : comment on construit concrètement ce système, étape par étape.

Chapitre 2 – Conception et Réalisation du Système

I. Choix du modèle

Pour ce projet, plusieurs modèles open-source récents ont été considérés, notamment **LLaMA 3.2** et **Gemma 3**, qui se distinguent par leurs performances et leurs tailles variées. Ces modèles sont disponibles en différentes versions, adaptées à divers besoins en termes de ressources matérielles et de capacités.

Modèle	Paramètres (en milliards)	Taille en mémoire	Remarques principales
LLaMA 3.2:latest	3	~2 GB	Modèle complet offrant une bonne capacité à gérer un contexte long avec une précision élevée.
Gemma 3:latest	4	~3.3 GB	Plus grand modèle, performances accrues mais nécessite plus de ressources.
LLaMA 3.2:1B	1	~1.3 GB	Version compacte adaptée aux machines avec ressources limitées, particulièrement efficace sur CPU.
Gemma 3:1B	1	~815 MB	Modèle léger, taille mémoire très réduite, mais potentiellement moins performant que LLaMA 3.2:1B.

1. Critères de sélection

- ✓ **Ressources matérielles** : Étant donné que le projet s'exécute principalement sur un PC local **sans GPU dédié**, la capacité à fonctionner efficacement sur **CPU avec une faible mémoire** est primordiale.
- ✓ **Performance vs Taille** : Les versions « latest » (3-4 milliards de paramètres) offrent des capacités supérieures en compréhension contextuelle et en raisonnement complexe, mais leur taille et leur besoin en calcul sont trop élevés pour un déploiement fluide sur un PC sans GPU.
- ✓ **Compatibilité avec le contexte d'usage** : Le modèle doit bien s'intégrer dans un cadre de génération augmentée par récupération (RAG) avec des données textuelles extraites de formats PDF, DOCX et TXT.

2. Choix final

Par conséquent, le modèle **LLaMA 3.2:1B** a été retenu pour ce projet, car il propose le meilleur compromis entre :

- Une taille mémoire adaptée à un PC standard (environ 1.3 GB),
- Une capacité raisonnable à traiter des requêtes en temps réel sur CPU,
- Une performance satisfaisante pour des tâches classiques de génération de texte,
- Une intégration simplifiée via la plateforme **Ollama**.

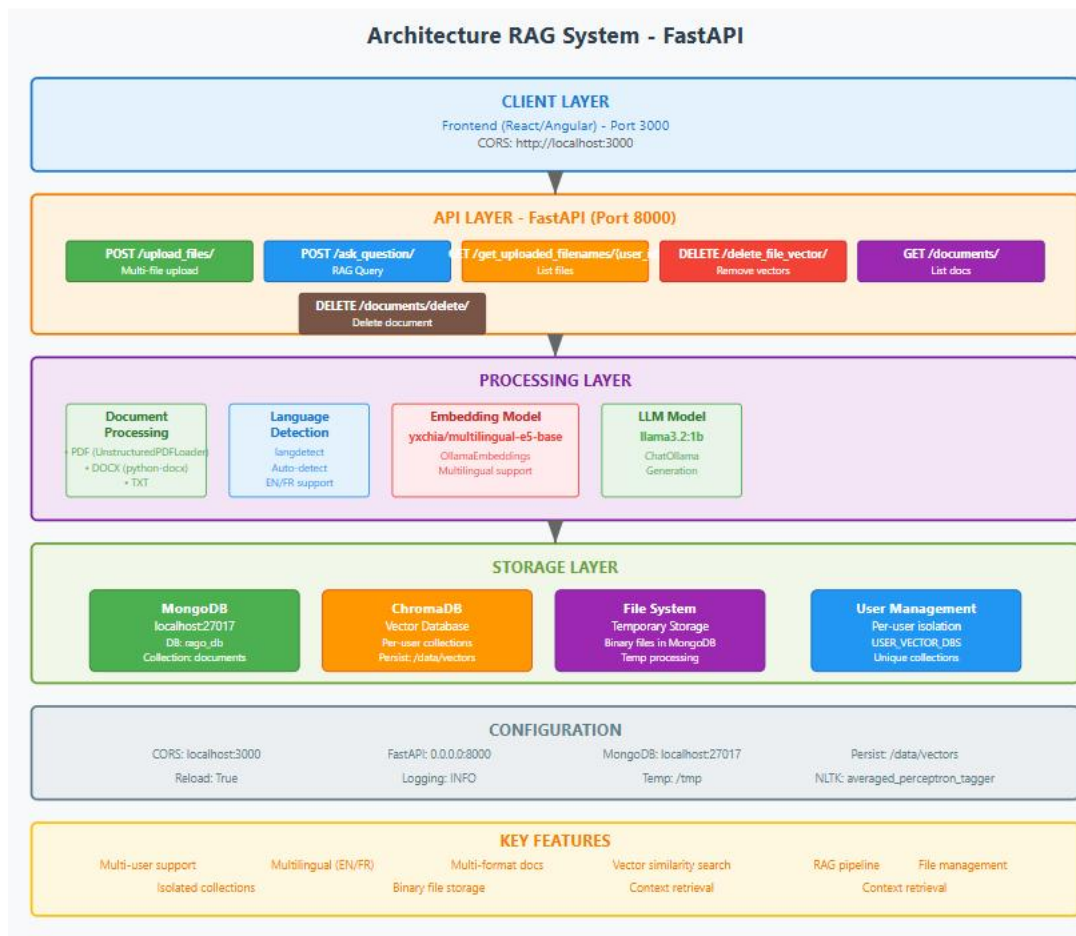
Ainsi, bien que des modèles plus grands comme **LLaMA 3.2:latest** ou **Gemma 3:latest** offrent des performances supérieures, ils ne correspondent pas aux contraintes matérielles du déploiement local. **Gemma 3:1B** est également très léger, mais moins performant que LLaMA 3.2:1B selon les benchmarks disponibles.

II. Partie Backend (Serveur et logique RAG)

Le système backend a été élaboré à l'aide de FastAPI et représente le cœur du traitement des documents et de la production de réponses au sein de l'architecture RAG (Retrieval-Augmented Generation). Elle s'occupe de la réception des fichiers des utilisateurs, de l'extraction et de l'analyse de leur contenu, de l'indexation sémantique et, en fin de compte, de la production de réponses grâce au modèle LLaMA 3.2:1b, exécuté localement à l'aide d'Ollama.

1. Architecture générale du backend

a. Diagramme d'architecture système



Le diagramme d'architecture système représente la structure globale de votre application et montre comment les différents composants interagissent entre eux.

Principales composantes

1. **Backend FastAPI** : Le cœur de l'application avec les endpoints principaux :

- **POST /upload_files/**

Permet d'envoyer des fichiers au serveur.

- **GET /get_uploaded_filenames/{user_id}**

Retourne la liste des fichiers envoyés par un utilisateur, stockés dans ChromaDB.

- **DELETE /delete_file_vector/**

Supprime les vecteurs associés à un fichier dans ChromaDB pour un utilisateur.

- **POST /ask_question/**

Permet de poser une question sur les documents envoyés et retourne une réponse.

GET /documents/

Affiche tous les documents enregistrés dans MongoDB.

- DELETE /documents/delete/

Supprime un ou plusieurs documents de la base de données MongoDB.

2. **Traitement des documents** : Composants responsables de l'extraction et de la préparation des données :

- a) **Extracteurs de texte spécifiques** adaptés aux formats PDF, DOCX et TXT .
- b) **Traitement léger du texte extrait** via une fonction de nettoyage simple (par exemple, suppression d'espaces ou caractères inutiles).
- c) **Stockage du contenu binaire du fichier** dans MongoDB pour conserver l'original.
- d) **Stockage du texte brut extrait** dans MongoDB avec ses métadonnées (nom de fichier, utilisateur, date).
- e) **Insertion du texte complet** dans une base vectorielle Chroma propre à chaque utilisateur pour la recherche vectorielle.

3. **Stockage des données** : Deux bases de données distinctes avec des rôles complémentaires :

- **MongoDB** : stockage du fichier original en binaire, du texte extrait, des métadonnées et de la date d'upload.
- **ChromaDB** (une base vectorielle) : stockage des embeddings (vecteurs) du texte extrait, indexés par utilisateur.

4. **Modèles IA** : Les moteurs d'intelligence artificielle :

- OllamaEmbeddings (yxchia/multilingual-e5-base) pour convertir le texte en vecteurs
- Ollama LLM (Llama 3.2:1b) pour générer des réponses intelligentes

Flux de données

Le client communique avec FastAPI, qui centralise toutes les opérations.

Lors de l'upload d'un fichier, le traitement suit ce chemin :

**upload → extraction de texte → sauvegarde binaire + texte dans MongoDB
→ génération et stockage des embeddings dans ChromaDB.**

Lorsqu'une question est posée, FastAPI :

- détecte la langue de la question,
- récupère les documents vectoriels pertinents via ChromaDB,
- construit un prompt adapté à la langue détectée,
- génère la réponse via Llama 3.2,
- retourne la réponse avec le contexte utilisé.

b. Diagramme de séquence



Le diagramme de séquence montre la chronologie des interactions entre les composants lors des deux opérations principales : l'upload de fichiers et la réponse aux questions.

Processus d'upload de fichier

- Le client envoie un ou plusieurs fichiers via une requête POST à l'endpoint `/upload_files/`.
- Le texte est extrait selon le format du fichier (PDF, DOCX, TXT).
- Le contenu binaire du fichier ainsi que le texte complet extrait sont sauvegardés dans MongoDB avec les métadonnées.
- Le texte est converti en vecteur et stocké dans une base ChromaDB propre à l'utilisateur.
- Une réponse de confirmation est renvoyée au client pour chaque fichier traité.

Processus de question-réponse

1. Le client envoie une question via une requête POST à l'endpoint `/ask_question`.
2. Le système détecte la langue de la question et prépare un prompt adapté (en français ou en anglais).
3. Les fragments de texte les plus pertinents sont récupérés depuis la base ChromaDB propre à l'utilisateur.
4. Ces fragments sont utilisés comme contexte dans le prompt envoyé au modèle LLM (Llama 3.2:1b).
5. La réponse générée est renvoyée au client, accompagnée du contexte utilisé.

Points forts

- ✧ **Flux structuré** : Chaque étape du processus est bien définie, de la réception de la question à la réponse.
- ✧ **Traitement intelligent** : Le prompt est automatiquement adapté à la langue de la question.
- ✧ **Récupération efficace** : Le système sélectionne uniquement les fragments pertinents pour améliorer la qualité des réponses.

2. Technologies et bibliothèques utilisées

- ❖ **FastAPI** : framework web léger et performant utilisé pour exposer les routes HTTP asynchrones.

Utilisation dans le projet:

-Définit les endpoints `/upload_files/`, `/ask_question/`,
`get/uploaded_filenames`, `/delete_file_vector`, `/documents/`,
`/documents/delete`.

-Gère la réception et le traitement des fichiers en entrée.

-Traite les requêtes utilisateur pour la récupération des documents et la génération des réponses.

- ❖ **LangChain** : bibliothèque de haut niveau pour la construction de pipelines NLP utilisant des LLM, intégrant loaders, splitters, retrievers, prompts et LLMs.

Utilisation dans le projet :

- Gestion des fichiers avec UnstructuredPDFLoader.
- Découpage intelligent du texte .
- Création des embeddings avec OllamaEmbeddings.

- ❖ **Ollama + LLaMA 3** : plateforme locale permettant de lancer un LLM sans dépendre du cloud, avec un modèle open-source de Meta.

Utilisation dans le projet :

- Création des embeddings via OllamaEmbeddings(model="yxchia/multilingual-e5-base").
- Génération de réponses avec ChatOllama(model="llama3.2:1b").

- ❖ **MongoDB** : base de données NoSQL utilisée pour stocker le contenu brut, les segments de texte et les métadonnées de chaque document.

Utilisation dans le projet :

- Stockage des documents traités avec le texte extrait et le nom du fichier.
- Requête pour lister les documents disponibles via l'endpoint /documents/

- ❖ **Chroma** : base vectorielle locale pour l'indexation sémantique des chunks à l'aide d'embeddings.

Utilisation dans le projet:

- Création d'une base de vecteurs persistante dans data/vectors/.
- Stockage des embeddings pour permettre la récupération contextuelle d'information

- ❖ **Pymongo, Docx, UnstructuredPDFLoader** : pour le chargement, la lecture, et le traitement de différents formats de fichiers.

Bibliothèque	Utilité
pymongo	Connexion et manipulation de MongoDB
docx	Extraction de texte depuis des fichiers .docx
shutil	Gestion des fichiers temporaires
tempfile	Création de fichiers temporaires pour le traitement
logging	Gestion des logs pour le suivi des erreurs et des actions

3. Endpoints API principaux

Endpoint	Méthode	Description
<code>/upload_files/</code>	POST	Téléverse et traite des fichiers
<code>/ask_question/</code>	POST	Répond à une question basée sur les documents
<code>/get_uploaded_filenames/{user_id}</code>	GET	Liste les fichiers d'un utilisateur
<code>/delete_file_vector/</code>	DELETE	Supprime les vecteurs d'un fichier
<code>/documents/</code>	GET	Liste les documents d'un utilisateur
<code>/documents/delete/</code>	DELETE	Supprime un document

a. Endpoint `/upload_files/`

Etablit un point d'accès FastAPI permettant aux utilisateurs d'envoyer plusieurs fichiers, créant ou récupérant un mutex pour empêcher l'accès simultané. Le système vérifie si un vecteur Chroma est déjà lié à l'utilisateur, et en cas contraire, un orchestrateur multi-agent est réajusté. Chaque fichier est traité individuellement, le texte est extrait, nettoyé puis divisé en segments de 2000 tokens. L'intégralité du texte et du fichier binaire est conservée dans MongoDB.

b. Endpoint `/ask_question/`

Etablit un point d'accès FastAPI pour permettre aux utilisateurs de poser des questions concernant les documents précédemment téléchargés. La classe `QuestionRequest` sert de système de validation, nécessitant un identifiant d'utilisateur et une question. Le serveur vérifie si le système multi-agent est initialisé, et si ce n'est pas le cas, il examine si l'utilisateur dispose d'un index valide. Si tout est en ordre, la question est transmise à un orchestrateur multi-agent pour obtenir une réponse précise et organisée.

c. Endpoint `/get_uploaded_filenames/{user_id}`

Etablit un point de terminaison FastAPI accessible par GET à l'URL `/get_uploaded_filenames/{user_id}`, permettant de récupérer la liste des fichiers associés à `user_id`, ainsi que leurs noms et tailles. Le système vérifie si l'utilisateur possède une base vectorielle dans `USER_VECTOR_DBS`, puis utilise un verrou pour garantir une lecture sécurisée des données. Il extrait ensuite les métadonnées des documents stockés et renvoie les noms de fichiers uniques ainsi que leurs tailles respective.

d. Endpoint `/delete_file_vector/`

La fonction établit un point d'accès FastAPI nommé DELETE, autorisant les

utilisateurs à supprimer tous les vecteurs liés à un fichier spécifique dans leur base de vecteurs. Il vérifie si l'utilisateur possède un vecteur de base dans USER_VECTOR_DBS, initialise un verrou et extrait les identifiants des vecteurs ainsi que les métadonnées. Si aucun vecteur n'est trouvé, une erreur 404 est déclenchée, et les vecteurs sont supprimés par lots de 100 pour améliorer la performance.

e. Endpoint /documents/

Détermine un point de terminaison FastAPI accessible via GET à /documents/, qui récupère une liste de documents envoyés par un utilisateur spécifique, identifié par user_id. Elle interroge la base de données MongoDB pour récupérer des fichiers, convertit le contenu binaire en Base64 pour la transmission JSON et formate les horodatages en ISO pour une meilleure lisibilité. Si aucun document n'est trouvé, une erreur 404 est déclenchée.

f. Endpoint /documents/delete/

L'endpoint SUPPRIMER clôt l'ensemble du cycle de vie des documents en les éliminant du système. Il accepte une requête JSON contenant l'ID de l'utilisateur et le nom du fichier. Cette méthode garantit qu'aucun document orphelin ne reste dans le système. Il englobe la validation des paramètres, une journalisation détaillée et une gestion des erreurs robuste.

2. Pipeline du traitement du document

Cette fonction établit un point d'accès FastAPI pour supprimer un document précédemment enregistré dans la base de données MongoDB. Une requête JSON est nécessaire, comprenant deux champs : user_id et filename. Si les champs ne sont pas remplis, une erreur 400 est renvoyée. Si aucun document n'est trouvé, une erreur 404 est renvoyée. En cas de succès, un message de confirmation est envoyé.

Importation des bibliothèques:

```

import os
import re
from fastapi import FastAPI, File, UploadFile, HTTPException, Form, Query, Request
import shutil
import tempfile
import logging
import json
import docx
import pymongo
from langchain_community.document_loaders import UnstructuredPDFLoader
from langchain_ollama import OllamaEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_ollama import ChatOllama
from langchain_core.runnables import RunnablePassthrough
import nltk
nltk.download('averaged_perceptron_tagger')
from datetime import datetime
import traceback
import base64
import numpy as np
from langdetect import detect
from pydantic import BaseModel
from typing import Dict, List, Optional, Annotated, TypedDict
from bson.binary import Binary
from langchain.text_splitter import TokenTextSplitter
from sentence_transformers import CrossEncoder
from fastapi.middleware.cors import CORSMiddleware
import asyncio
from threading import Lock

# ===== IMPORTS POUR LE SYSTÈME MULTI-AGENT =====
from langchain.tools import BaseTool
from langchain_core.messages import BaseMessage, HumanMessage, AIMessage, SystemMessage
from langgraph.graph import StateGraph, END
from langchain.schema import Document
from enum import Enum

```

Le bloc d'imports initialise toutes les bibliothèques nécessaires au backend FastAPI d'une application intelligente de traitement documentaire. Il inclut des modules standards pour la gestion des fichiers (os, shutil, tempfile), des exceptions (traceback, logging), la manipulation JSON et base64, ainsi que des opérations asynchrones et concurrentes (asyncio, Lock). Côté web, FastAPI est utilisé avec ses outils pour gérer les requêtes, fichiers, formulaires, ainsi que CORS pour autoriser les appels cross-origin. Pour le traitement de documents, docx et pdfplumber (utilisé ailleurs) permettent d'extraire le texte et les tableaux. L'application s'appuie massivement sur l'écosystème LangChain avec des composants comme UnstructuredPDFLoader, OllamaEmbeddings, ChatOllama, Chroma pour les bases vectorielles, et des prompts structurés (ChatPromptTemplate) couplés à des parseurs (StrOutputParser). L'orchestration multi-agent est gérée via langgraph, avec StateGraph, des types de messages (HumanMessage, AIMessage, etc.), et la classe BaseTool pour encapsuler des outils spécialisés. Pour le traitement NLP, nltk est utilisé (avec téléchargement du tagger grammatical), langdetect pour détecter la langue des requêtes, et TokenTextSplitter pour découper efficacement le texte. Enfin, pymongo et Binary assurent la persistance des fichiers et des métadonnées dans une base MongoDB.

Configuration d' environnement:

```

# Configuration du logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ===== CONFIGURATION DE L'APPLICATION =====
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Connexion à MongoDB
mongo_client = pymongo.MongoClient("mongodb://localhost:27017/")
db = mongo_client["rago_db"]
docs_collection = db["documents"]
db1 = mongo_client.llm_evaluation
collection1 = db1.qa_tests

# Configuration des répertoires
PERSIST_DIRECTORY = os.path.join("data", "vectors")
USER_VECTOR_DBS: Dict[str, Chroma] = {}
USER_LOCKS: Dict[str, Lock] = {}

# Modèle re-ranker
reranker = CrossEncoder("bAAI/bge-reranker-large", max_length=512)

```

Ce bloc configure l'environnement initial de l'application FastAPI. D'abord, le logging est activé au niveau INFO pour tracer les événements importants dans la console, à l'aide d'un logger nommé d'après le module courant (`__name__`). Ensuite, une instance FastAPI est créée et enrichie avec un middleware CORS qui autorise les requêtes provenant du frontend en `http://localhost:3000`, utile pour le développement local. La connexion à MongoDB est établie via `pymongo` sur `localhost:27017`, avec deux bases : `rago_db` pour stocker les documents (`docs_collection`) et `llm_evaluation` pour les tests qualité de LLM (`collection1`). Un répertoire `data/vectors` est défini pour persister les bases vectorielles localement, et deux dictionnaires globaux maintiennent en mémoire les bases Chroma des utilisateurs (`USER_VECTOR_DBS`) et des verrous par utilisateur (`USER_LOCKS`) pour éviter les accès concurrents. Enfin, un modèle de reranking nommé `bAAI/bge-reranker-large` est chargé via `CrossEncoder`, afin d'améliorer la pertinence des documents retournés par recherche vectorielle.

Définition de la structure formelle du système multi-agent

```
# ===== DÉFINITION DES AGENTS =====

class AgentType(Enum):
    DOCUMENT_RETRIEVER = "document_retriever"
    CONTENT_ANALYZER = "content_analyzer"
    RESPONSE_SYNTHESIZER = "response_synthesizer"
    QUALITY_CONTROLLER = "quality_controller"

class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], "The messages in the conversation"]
    question: str
    user_id: str
    retrieved_docs: List[Document]
    analysis_result: str
    synthesized_response: str
    quality_score: float
    final_response: str
    context_used: str
```

Ce bloc définit la structure formelle du système multi-agent. La classe `AgentType`, une énumération (`Enum`), liste les rôles disponibles pour les agents : récupération de documents (`DOCUMENT_RETRIEVER`), analyse de contenu (`CONTENT_ANALYZER`), synthèse de réponse (`RESPONSE_SYNTHESIZER`) et évaluation qualité (`QUALITY_CONTROLLER`). Ensuite, `AgentState` est un `TypedDict` typé avec `Annotated`, représentant l'état partagé que chaque agent lit et modifie dans le pipeline. Il contient notamment :

- la liste des messages échangés (`messages`),
- la question posée (`question`),
- l'identifiant utilisateur (`user_id`),
- les documents récupérés (`retrieved_docs`),
- le résultat d'analyse (`analysis_result`),
- la réponse synthétisée (`synthesized_response`),
- le score qualité (`quality_score`),
- la réponse finale (`final_response`),
- et le contexte utilisé (`context_used`).

Cette structure unifiée permet de faire circuler l'information entre agents dans un graphe de traitement défini par `LangGraph`.

Outils personnalisés des agents


```
# ===== OUTILS PERSONNALISÉS POUR LES AGENTS =====

class DocumentRetrievalTool(BaseTool):
    name: str = "document_retrieval"
    description: str = "Récupère les documents pertinents depuis la base vectorielle"

    def __init__(self, vector_dbs: Dict[str, Chroma]):
        super().__init__()
        self.vector_dbs = vector_dbs

    def _run(self, query: str, user_id: str) -> List[Document]:
        if user_id not in self.vector_dbs:
            return []

        vector_db = self.vector_dbs[user_id]
        retriever = vector_db.as_retriever(search_kwargs={"k": 8})
        docs = retriever.get_relevant_documents(query)

        if docs:
            # Utilisation du re-ranker pour améliorer la pertinence
            pairs = [(query, doc.page_content) for doc in docs]
            scores = reranker.predict(pairs)
            sorted_docs = [doc for _, doc in sorted(zip(scores, docs), key=lambda x: x[0], reverse=True)]
            return sorted_docs[:5]

        return []
```

Cette classe `DocumentRetrievalTool`, héritée de `BaseTool`, est un outil personnalisé utilisé par l'agent de récupération documentaire. Elle sert à interroger la base vectorielle Chroma associée à un utilisateur donné (`user_id`) pour récupérer les documents les plus pertinents face à une requête (`query`). Lors de l'appel de la méthode `_run`, l'outil vérifie que l'utilisateur a bien une base vectorielle, puis utilise `as_retriever(k=8)` pour extraire jusqu'à 8 documents similaires. Si des documents sont trouvés, ils sont ensuite rerankés par pertinence à l'aide du modèle `bAAI/bge-reranker-large`, qui évalue chaque paire (`query`, contenu du document) et renvoie un score de similarité. Les documents sont triés par score décroissant, et seuls les 5 meilleurs sont retournés. Ce reranking garantit que les documents fournis à l'agent suivant sont réellement les plus proches contextuellement de la question posée.

```
class ContentAnalysisTool(BaseTool):
    name: str = "content_analysis"
    description: str = "Analyse le contenu des documents pour extraire les informations pertinentes"

    def __init__(self, llm):
        super().__init__()
        self.llm = llm

    def _run(self, documents: List[Document], question: str) -> str:
        if not documents:
            return "Aucun document à analyser"

        analysis_prompt = ChatPromptTemplate.from_template("""
        En tant qu'expert en analyse de contenu, analysez les documents suivants pour répondre à la question.

        Documents:
        {documents}

        Question: {question}

        Fournissez une analyse structurée qui identifie:
        1. Les informations clés pertinentes
        2. Les relations entre les concepts
        3. Les éléments manquants éventuels
        4. Un résumé concis des points importants

        Analyse:
        """)

        docs_text = "\n\n---\n\n".join([doc.page_content for doc in documents])

        chain = analysis_prompt | self.llm | StrOutputParser()
        return chain.invoke({"documents": docs_text, "question": question})
```

La classe `ContentAnalysisTool`, dérivée de `BaseTool`, est conçue pour effectuer une analyse

sémantique approfondie d'un ensemble de documents à l'aide d'un LLM. Elle reçoit deux entrées principales : la liste des documents (objets Document) et la question posée par l'utilisateur. Si aucun document n'est fourni, elle retourne une phrase par défaut signalant l'absence de contenu. Sinon, elle construit dynamiquement un prompt (ChatPromptTemplate) rédigé en français, demandant à un agent "expert en analyse de contenu" d'examiner les documents pour en extraire :

- les informations clés pertinentes,
- les relations entre les concepts,
- les éléments manquants éventuels,
- un résumé concis.

Le contenu des documents est concaténé avec des séparateurs pour contextualisation, puis injecté dans la chaîne Prompt → LLM → OutputParser, exécutée avec `.invoke(...)`. Le résultat est une string textuelle contenant l'analyse structurée, qui servira à l'agent de synthèse pour formuler une réponse cohérente.

Agents spécialisés

```
# ===== AGENTS SPÉCIALISÉS =====
class DocumentRetrieverAgent:
    def __init__(self, vector_dbs: Dict[str, Chroma]):
        self.tool = DocumentRetrievalTool(vector_dbs)

    async def execute(self, state: AgentState) -> AgentState:
        logger.info("📖 Agent de récupération - Recherche de documents pertinents")
        retrieved_docs = self.tool._run(state["question"], state["user_id"])
        state["retrieved_docs"] = retrieved_docs
        return state

class ContentAnalyzerAgent:
    def __init__(self):
        self.tool = ContentAnalysisTool(ChatOllama(model="llama3.2:1b"))

    async def execute(self, state: AgentState) -> AgentState:
        logger.info("🔍 Agent d'analyse - Traitement du contenu")
        result = self.tool._run(state["retrieved_docs"], state["question"])
        state["analysis_result"] = result
        return state
```

class DocumentRetrieverAgent: Cet agent est responsable de la récupération des documents pertinents à partir de la base vectorielle pour un utilisateur donné. Il reçoit en paramètre un dictionnaire `vector_dbs` où chaque clé est un `user_id` et la valeur est une base vectorielle (Chroma) contenant les embeddings des documents de cet utilisateur.

Il initialise l'outil `DocumentRetrievalTool`, un outil personnalisé qui encapsule la logique de recherche vectorielle. `execute` est la méthode asynchrone utilisée dans le graphe multi-agent.

Elle prend en entrée un `AgentState` (un dictionnaire typé représentant l'état partagé entre les agents).

Elle appelle `_run` de l'outil `DocumentRetrievalTool`, qui utilise :

la question (`state["question"]`)

l'identifiant de l'utilisateur (`state["user_id"]`)

Elle stocke les documents pertinents retrouvés dans `state["retrieved_docs"]`.

class ContentAnalyzerAgent: Cet agent est responsable d'analyser le contenu des documents récupérés pour extraire des informations utiles et répondre à la question.

Il initialise un outil `ContentAnalysisTool` basé sur un modèle LLM local appelé "llama3.2:1b" (via ChatOllama).

Cet outil est chargé de formuler une analyse en langage naturel à partir des documents et de la question. `execute` prend aussi un `AgentState`.

Elle exécute l'analyse via `_run` de `ContentAnalysisTool`, en lui passant :

les documents récupérés

la question de l'utilisateur

Le résultat de l'analyse est stocké dans `state["analysis_result"]`.

```
class ResponseSynthesizerAgent:
    async def execute(self, state: AgentState) -> AgentState:
        logger.info("👉 Agent de synthèse - Génération de la réponse")

        lang = detect(state["question"])

        if lang == "en":
            synthesis_prompt = ChatPromptTemplate.from_template("""
                Based on the analysis provided, synthesize a comprehensive and accurate response to the user's question.

                Analysis: {analysis}
                Question: {question}

                Provide a clear, concise, and well-structured response in English.
                If the information is insufficient, state this clearly.

                Response:
                """)
        else:
            synthesis_prompt = ChatPromptTemplate.from_template("""
                Basé sur l'analyse fournie, synthétisez une réponse complète et précise à la question de l'utilisateur.

                Analyse: {analysis}
                Question: {question}

                Fournissez une réponse claire, concise et bien structurée en français.
                Si l'information est insuffisante, indiquez-le clairement.

                Réponse:
                """)

        chain = synthesis_prompt | self.llm | StrOutputParser()
        response = chain.invoke({
            "analysis": state["analysis_result"],
            "question": state["question"]
        })

        state["synthesized_response"] = response
        return state
```

La classe `ResponseSynthesizerAgent` est un agent dont le rôle est de générer une réponse finale à la question de l'utilisateur en se basant sur l'analyse précédente. Lors de son initialisation, il instancie un modèle LLM local (llama3.2:1b) via `ChatOllama`. Dans sa méthode asynchrone `execute`, l'agent commence par détecter la langue de la question (anglais ou français). En fonction de cette langue, il sélectionne un prompt de synthèse approprié, conçu pour guider le LLM à produire une réponse claire, concise et bien structurée. Ce prompt inclut l'analyse et la question, et est ensuite injecté dans une chaîne composée du prompt, du modèle LLM et d'un parseur de sortie. La réponse générée est stockée dans `state["synthesized_response"]`, prête à être évaluée ou retournée.

```

class QualityControllerAgent:
    def __init__(self):
        self.llm = ChatOllama(model="llama3.2:1b")

    async def execute(self, state: AgentState) -> AgentState:
        logger.info("🔍 Agent de contrôle qualité - Évaluation de la réponse")

        quality_prompt = ChatPromptTemplate.from_template("""
        Évaluez la qualité de cette réponse sur une échelle de 0 à 1:

        Question: {question}
        Réponse: {response}

        Critères d'évaluation:
        - Pertinence (répond-elle à la question?)
        - Précision (les informations sont-elles correctes?)
        - Complétude (la réponse est-elle suffisamment détaillée?)
        - Clarté (la réponse est-elle bien structurée?)

        Retournez uniquement un nombre entre 0 et 1, puis une brève justification.
        Format: SCORE: 0.X
        JUSTIFICATION: ...
        """)

        chain = quality_prompt | self.llm | StrOutputParser()
        evaluation = chain.invoke({
            "question": state["question"],
            "response": state["synthesized_response"]
        })

        # Extraction du score
        try:
            score_line = [line for line in evaluation.split('\n') if line.startswith('SCORE:')]
            score = float(score_line[0].split(':')[1].strip())
        except:
            score = 0.5 # Score par défaut

        state["quality_score"] = score

        # Si le score est trop bas, on peut décider de reformuler
        if score < 0.6:
            logger.warning(f"⚠️ Score de qualité faible: {score}")
            # Ici on pourrait implémenter une logique de reformulation

        state["final_response"] = state["synthesized_response"]
        return state

```

La classe `QualityControllerAgent` est un agent chargé d'évaluer la qualité de la réponse générée par le système. Lors de son initialisation, elle utilise le modèle `llama3.2:1b` via `ChatOllama`. Dans sa méthode `execute`, elle construit un prompt qui demande explicitement au modèle de noter la réponse sur une échelle de 0 à 1, selon quatre critères : pertinence, précision, complétude et clarté. Ce prompt est injecté dans une chaîne (prompt | llm | StrOutputParser) avec la question et la réponse en entrée. Le modèle retourne un texte contenant une ligne de score (SCORE: 0.X) suivie d'une justification. L'agent extrait ce score, l'ajoute à `state["quality_score"]`, et copie la réponse dans `state["final_response"]`. Si le score est jugé trop faible (inférieur à 0.6), un avertissement est loggé, laissant la possibilité d'implémenter une reformulation plus tard.

Orchestrateur multi-agents

```
# ===== ORCHESTRATEUR MULTI-AGENT =====
```

```
class MultiAgentOrchestrator:
    def __init__(self, vector_dbs: Dict[str, Chroma]):
        self.retriever_agent = DocumentRetrieverAgent(vector_dbs)
        self.analyzer_agent = ContentAnalyzerAgent()
        self.synthesizer_agent = ResponseSynthesizerAgent()
        self.quality_agent = QualityControllerAgent()

        # Création du graphe de workflow
        self.workflow = self._create_workflow()

    def _create_workflow(self):
        # Définition des nœuds du workflow
        workflow = StateGraph(AgentState)

        workflow.add_node("retriever", self.retriever_agent.execute)
        workflow.add_node("analyzer", self.analyzer_agent.execute)
        workflow.add_node("synthesizer", self.synthesizer_agent.execute)
        workflow.add_node("quality_controller", self.quality_agent.execute)

        # Définition des transitions
        workflow.set_entry_point("retriever")
        workflow.add_edge("retriever", "analyzer")
        workflow.add_edge("analyzer", "synthesizer")
        workflow.add_edge("synthesizer", "quality_controller")
        workflow.add_edge("quality_controller", END)

        return workflow.compile()
```

```
async def process_question(self, user_id: str, question: str) -> Dict:
    logger.info(f"🚀 Démarrage du processus multi-agent pour: {question}")

    initial_state = AgentState(
        messages=[HumanMessage(content=question)],
        question=question,
        user_id=user_id,
        retrieved_docs=[],
        analysis_result="",
        synthesized_response="",
        quality_score=0.0,
        final_response="",
        context_used=""
    )

    # Exécution du workflow
    final_state = await self.workflow.ainvoke(initial_state)

    # Préparation du contexte utilisé
    context_parts = [doc.page_content.strip() for doc in final_state["retrieved_docs"][:3]]
    context_used = "\n\n---\n\n".join(context_parts) if context_parts else "Aucun contexte trouvé"

    return {
        "question": question,
        "response": final_state["final_response"],
        "context_used": context_used,
        "quality_score": final_state["quality_score"],
        "documents_found": len(final_state["retrieved_docs"]),
        "analysis_summary": final_state["analysis_result"][:200] + "..." if len(final_state["analysis_result"]) > 200 else final_state["analysis_result"]
    }
```

La classe `MultiAgentOrchestrator` orchestre l'exécution coordonnée de plusieurs agents spécialisés pour traiter une question utilisateur. À l'initialisation, elle crée quatre agents : un pour la récupération des documents (`DocumentRetrieverAgent`), un pour l'analyse du contenu (`ContentAnalyzerAgent`), un pour la synthèse de réponse (`ResponseSynthesizerAgent`), et un pour le contrôle qualité (`QualityControllerAgent`). Elle définit ensuite un graphe de workflow (`StateGraph`) où chaque nœud représente l'exécution d'un agent, et les transitions suivent l'ordre logique : récupération → analyse → synthèse → évaluation. Lorsqu'une question est posée via `process_question`, l'orchestrateur construit un état initial avec les métadonnées nécessaires (question, `user_id`, etc.), puis déclenche l'exécution du graphe. À la fin, il extrait les trois premiers documents utilisés pour constituer le contexte, compile les résultats, le score de qualité et l'analyse, puis retourne une réponse complète prête à être affichée à l'utilisateur.

Fonctions utilitaires


```

from docx import Document
def extract_tables_from_docx(file_path, filename):
    doc = Document(file_path)
    all_tables_text = []
    for table in doc.tables:
        for row in table.rows:
            row_text = " | ".join(cell.text.strip() for cell in row.cells)
            all_tables_text.append(row_text)
    full_table_text = "\n".join(all_tables_text)
    context_header = f"Contenu extrait des tables du document {filename}.\n\n"
    return [context_header + full_table_text]

def generate_unique_filename(filename, user_id):
    name, ext = os.path.splitext(filename)
    i = 1
    new_filename = filename
    while docs_collection.find_one({"user_id": user_id, "filename": new_filename}):
        new_filename = f"{name} ({i}){ext}"
        i += 1
    return new_filename

def format_size(bytes_size: int) -> str:
    for unit in ["B", "KB", "MB"]:
        if bytes_size < 1024:
            return f"{bytes_size:.2f} {unit}"
        bytes_size /= 1024
    return f"{bytes_size:.2f} MB"

class FileInfo(BaseModel):
    filename: str
    size: str

```

La fonction `extract_tables_from_docx` lit les tableaux d'un document Word (.docx), extrait chaque ligne en joignant les cellules avec des barres verticales, puis renvoie le contenu tabulaire sous forme de texte avec un en-tête précisant le nom du fichier. `generate_unique_filename` garantit qu'un nom de fichier soit unique pour chaque utilisateur en vérifiant dans MongoDB s'il existe déjà un document portant ce nom, et en ajoutant un suffixe numérique si nécessaire. `format_size` convertit une taille en octets en une chaîne lisible avec les unités B, KB ou MB, utilisée pour afficher la taille des fichiers. Enfin, la classe `FileInfo`, définie avec Pydantic, structure les métadonnées des fichiers en précisant leur nom et leur taille au format lisible, utile pour les réponses d'API.

Endpoints

```

@app.post("/upload_files/")
async def upload_files(
    files: List[UploadFile] = File(...),
    user_id: str = Form(...)
):
    global orchestrator

    if user_id not in USER_LOCKS:
        USER_LOCKS[user_id] = Lock()
    lock: Lock = USER_LOCKS[user_id]

    with lock:
        results = []
        embeddings = OllamaEmbeddings(model="yxchia/multilingual-e5-base")

        if user_id not in USER_VECTOR_DBS:
            user_persist_dir = os.path.join(PERSIST_DIRECTORY, user_id)
            os.makedirs(user_persist_dir, exist_ok=True)
            USER_VECTOR_DBS[user_id] = Chroma(
                embedding_function=embeddings,
                persist_directory=user_persist_dir
            )
            # Réinitialiser l'orchestrateur avec les nouvelles bases
            orchestrator = MultiAgentOrchestrator(USER_VECTOR_DBS)

        vector_db = USER_VECTOR_DBS[user_id]

        for file in files:
            logger.info(f"Traitement du fichier : {file.filename} pour user_id : {user_id}")
            temp_dir = None
            try:
                temp_dir = tempfile.mkdtemp()
                unique_filename = generate_unique_filename(file.filename, user_id)
                file_path = os.path.join(temp_dir, unique_filename)

```

```

                text = clean_text(text)

                splitter = TokenTextSplitter(chunk_size=2000, chunk_overlap=200, encoding_name="cl100k_base")
                chunks = splitter.split_text(text)

                chunk_metadata = [
                    {"filename": file.filename, "chunk_index": i, "size_bytes": os.path.getsize(file_path)}
                    for i in range(len(chunks))
                ]

                BATCH_SIZE = 100
                for i in range(0, len(chunks), BATCH_SIZE):
                    sub_chunks = chunks[i : i + BATCH_SIZE]
                    sub_metadata = chunk_metadata[i : i + BATCH_SIZE]
                    vector_db.add_texts(sub_chunks, metadata=sub_metadata)

                with open(file_path, "rb") as f:
                    file_binary = Binary(f.read())
                doc_data = {
                    "user_id": user_id,
                    "filename": unique_filename,
                    "text": text,
                    "file_data": file_binary,
                    "timestamp": datetime.now()
                }
                docs_collection.insert_one(doc_data)

                results.append({"status": "success", "filename": file.filename})

            except Exception as e:
                logger.error(f"Erreur sur {file.filename} : {traceback.format_exc()}")
                results.append({"status": "error", "filename": file.filename, "error": str(e)})

            finally:
                if temp_dir:
                    shutil.rmtree(temp_dir, ignore_errors=True)

        return results

```

Cette portion de code définit un endpoint FastAPI (/upload_files/) permettant à un utilisateur d'envoyer plusieurs fichiers (PDF, DOCX ou TXT) en précisant son identifiant (user_id). Lorsqu'un utilisateur envoie un fichier, le système crée ou récupère un verrou (mutex) pour éviter les accès concurrents à ses ressources. Ensuite, il vérifie si une base vectorielle Chroma est déjà associée à l'utilisateur ; sinon, elle est créée et liée à un modèle d'embedding (yxchia/multilingual-e5-base). Si nécessaire, l'orchestrateur multi-agent est réinitialisé pour prendre en compte cette nouvelle base. Chaque fichier est ensuite traité individuellement : il est temporairement enregistré, puis son texte est extrait selon son type (texte brut ou tableaux), nettoyé, et découpé en segments

(chunks) de 2000 tokens avec chevauchement. Ces segments, accompagnés de métadonnées (nom, index, taille...), sont insérés dans la base vectorielle. Par ailleurs, le texte entier et le fichier binaire sont stockés dans une collection MongoDB. À la fin de chaque traitement, le fichier temporaire est supprimé pour libérer de l'espace disque. Enfin, la fonction retourne un tableau résumant le statut de traitement de chaque fichier (succès ou erreur), ce qui permet à l'utilisateur de savoir quels fichiers ont été traités correctement.

```
class QuestionRequest(BaseModel):
    user_id: str
    question: str

@app.post("/ask_question/")
async def ask_question(data: QuestionRequest):
    """
    Endpoint principal utilisant le système multi-agent pour répondre aux questions
    """
    global orchestrator

    if not orchestrator:
        raise HTTPException(status_code=500, detail="Système multi-agent non initialisé")

    if data.user_id not in USER_VECTOR_DBS:
        raise HTTPException(status_code=400, detail="Aucun document chargé pour cet utilisateur.")

    try:
        logger.info(f"📩 Question reçue via système multi-agent: {data.question}")

        # Utilisation de l'orchestrateur multi-agent
        result = await orchestrator.process_question(data.user_id, data.question)

        logger.info(f"✅ Réponse générée avec score de qualité: {result['quality_score']}")

        return result

    except Exception as e:
        logger.error(f"Erreur dans le système multi-agent : {traceback.format_exc()}")
        raise HTTPException(status_code=500, detail="Erreur interne du système multi-agent.")
```

Cette partie du code définit un endpoint FastAPI (/ask_question/) permettant à un utilisateur d'envoyer une question en lien avec les documents qu'il a préalablement téléversés. La classe QuestionRequest sert de schéma de validation, imposant la présence d'un user_id (identifiant utilisateur) et d'une question (sous forme de chaîne de caractères). Lorsque l'utilisateur envoie sa requête, le serveur vérifie d'abord que le système multi-agent (orchestrator) est bien initialisé ; sinon, une erreur serveur (500) est levée. Ensuite, il s'assure que l'utilisateur concerné dispose bien d'une base vectorielle dans laquelle des documents ont été indexés. Si ce n'est pas le cas, une erreur (400) est renvoyée pour indiquer qu'aucun document n'a été chargé. Si tout est en ordre, la question est transmise à l'orchestrateur multi-agent qui active successivement plusieurs agents spécialisés (recherche documentaire, analyse de contenu, génération de réponse et évaluation de qualité) pour produire une réponse précise et structurée. Une fois la réponse générée, elle est renvoyée à l'utilisateur accompagnée d'un score de qualité. En cas d'erreur durant ce processus, l'exception est interceptée, enregistrée dans les logs, et une erreur 500 est renvoyée pour signaler une défaillance du système.


```

@app.get("/get_uploaded_filenames/{user_id}", response_model=List[FileInfo])
async def get_uploaded_filenames(user_id: str):
    if user_id not in USER_VECTOR_DBS:
        raise HTTPException(status_code=404, detail="Base vectorielle non trouvée pour cet utilisateur.")

    if user_id not in USER_LOCKS:
        USER_LOCKS[user_id] = Lock()
    lock: Lock = USER_LOCKS[user_id]

    lock.acquire()
    try:
        vector_db = USER_VECTOR_DBS[user_id]
        collection = vector_db._collection

        collection_data = collection.get(include=["metadatas"])
        metadatas = collection_data["metadatas"]

        files_info: List[FileInfo] = []
        seen_files = set()
        for meta in metadatas:
            if meta and "filename" in meta and meta.get("type") != "table":
                filename = meta["filename"]
                if filename not in seen_files:
                    size_bytes = meta.get("size_bytes", 0)
                    size_str = format_size(size_bytes)
                    files_info.append(FileInfo(filename=filename, size=size_str))
                    seen_files.add(filename)

        return files_info

    except Exception as e:
        logger.error(f"Erreur lors de la récupération des fichiers : {traceback.format_exc()}")
        raise HTTPException(status_code=500, detail="Erreur interne.")
    finally:
        lock.release()

```

Cette fonction définit un endpoint FastAPI accessible via la méthode GET à l'URL /get_uploaded_filenames/{user_id}. Elle permet de récupérer la liste des noms des fichiers que l'utilisateur identifié par user_id a précédemment téléversés et vectorisés, accompagnée de leur taille. Tout d'abord, le système vérifie que l'utilisateur dispose bien d'une base vectorielle dans USER_VECTOR_DBS, sinon une erreur 404 est renvoyée. Un verrou (Lock) est ensuite utilisé pour garantir une lecture sûre des données associées à l'utilisateur. La base vectorielle (Chroma) est interrogée pour extraire les métadonnées des documents stockés, et un filtre est appliqué pour exclure les entrées correspondant uniquement à des tableaux (type != "table"). Les noms de fichiers uniques sont collectés, ainsi que la taille de chaque fichier (formatée lisiblement), pour être retournés sous forme d'une liste d'objets FileInfo. Si une erreur survient, elle est enregistrée dans les logs, et une exception HTTP 500 est levée. Enfin, le verrou est systématiquement libéré grâce au bloc finally, assurant la bonne gestion de la concurrence même en cas d'échec.

```

@app.delete("/delete_file_vector/")
async def delete_file_vector(user_id: str = Query(...), filename: str = Query(...)):
    if user_id not in USER_VECTOR_DBS:
        raise HTTPException(status_code=404, detail="Base vectorielle non trouvée pour cet utilisateur.")

    if user_id not in USER_LOCKS:
        USER_LOCKS[user_id] = Lock()
    lock: Lock = USER_LOCKS[user_id]

    vector_db = USER_VECTOR_DBS[user_id]
    collection = vector_db._collection

    collection_data = collection.get(include=["metadatas"])
    all_ids = collection_data["ids"]
    all metas = collection_data["metadatas"]

    ids_to_delete = [
        doc_id
        for doc_id, meta in zip(all_ids, all metas)
        if meta and meta.get("filename") == filename
    ]

    if not ids_to_delete:
        raise HTTPException(status_code=404, detail="Aucun vecteur trouvé pour ce fichier.")

    with lock:
        try:
            BATCH_SIZE = 100
            total_deleted = 0

            for i in range(0, len(ids_to_delete), BATCH_SIZE):
                batch_ids = ids_to_delete[i:i + BATCH_SIZE]
                collection.delete(ids=batch_ids)
                total_deleted += len(batch_ids)
                logger.info(f"Supprimé batch {i//BATCH_SIZE + 1}: {len(batch_ids)} chunks")

            return {
                "status": "success",
                "deleted_vectors": total_deleted,
                "filename": filename
            }

```

Cette fonction définit un endpoint FastAPI accessible via la méthode DELETE à l'URL /delete_file_vector/, permettant à un utilisateur de supprimer tous les vecteurs associés à un fichier spécifique dans sa base vectorielle. Elle attend deux paramètres de requête : user_id (identifiant de l'utilisateur) et filename (nom du fichier à supprimer). Le système commence par vérifier si l'utilisateur possède bien une base vectorielle dans USER_VECTOR_DBS. Ensuite, il initialise un verrou pour garantir que l'opération de suppression ne soit pas perturbée par d'autres accès concurrents. La base vectorielle est ensuite parcourue pour extraire les identifiants (ids) des vecteurs et leurs métadonnées, dans le but de localiser ceux correspondant au nom de fichier ciblé. Si aucun vecteur n'est trouvé pour ce fichier, une erreur 404 est renvoyée. Sinon, les vecteurs sont supprimés par lots de 100 pour optimiser les performances et éviter les débordements mémoire. À chaque batch supprimé, un message de log est émis. En cas d'erreur pendant le processus, celle-ci est enregistrée et une erreur 500 est renvoyée à l'utilisateur. Enfin, si tout se passe bien, un message de succès est retourné, indiquant le nombre de vecteurs supprimés pour le fichier concerné.

```

@app.get("/documents/")
async def list_documents(user_id: str):
    try:
        documents_cursor = docs_collection.find(
            {"user_id": user_id},
            {"_id": 0, "filename": 1, "file_data": 1, "timestamp": 1}
        )

        documents = []
        for doc in documents_cursor:
            file_data_base64 = base64.b64encode(doc["file_data"]).decode("utf-8")
            documents.append({
                "filename": doc["filename"],
                "file_data": file_data_base64,
                "timestamp": doc["timestamp"].isoformat() if doc["timestamp"] else None
            })

        if not documents:
            raise HTTPException(status_code=404, detail="Aucun document trouvé pour cet utilisateur.")

        return {"documents": documents}

    except Exception as e:
        logger.error(f"Erreur lors de la récupération des documents: {e}")
        raise HTTPException(status_code=500, detail=f"Erreur: {str(e)}")

```

Cette fonction définit un endpoint FastAPI accessible via la méthode GET à l'URL /documents/, qui permet de récupérer la liste des documents téléversés par un utilisateur spécifique, identifié par le paramètre user_id. Elle interroge la base de données MongoDB (collection docs_collection) pour obtenir les fichiers correspondant à cet utilisateur, en sélectionnant uniquement les champs filename, file_data et timestamp (en excluant l'identifiant _id). Pour chaque document trouvé, le contenu binaire du fichier est converti en chaîne Base64 afin de pouvoir être renvoyé dans la réponse JSON, ce qui est nécessaire pour la transmission de données binaires via HTTP. L'horodatage (timestamp) est également formaté en ISO pour plus de lisibilité. Si aucun document n'est trouvé, une erreur 404 est levée pour indiquer que l'utilisateur ne possède aucun fichier. En cas d'erreur technique durant l'exécution, celle-ci est enregistrée dans les logs et une exception HTTP 500 est renvoyée avec un message d'erreur explicite.

```

@app.delete("/documents/delete/")
async def delete_document(request: Request):
    try:
        data = await request.json()
        user_id = data.get("user_id")
        filename = data.get("filename")

        if not user_id or not filename:
            logger.error("Paramètres manquants: user_id et filename sont requis")
            raise HTTPException(
                status_code=400,
                detail="user_id et filename sont requis"
            )

        logger.info(f"Tentative de suppression du document: {filename} pour user_id: {user_id}")

        result = docs_collection.delete_one({
            "user_id": user_id,
            "filename": filename
        })

        if result.deleted_count == 0:
            logger.warning(f"Document non trouvé: {filename} pour user_id: {user_id}")
            raise HTTPException(
                status_code=404,
                detail="Document non trouvé"
            )

        logger.info(f"Document supprimé avec succès: {filename}")
        return {
            "success": True,
            "message": f"Document {filename} supprimé avec succès"
        }

    except HTTPException as he:
        raise he
    except Exception as e:
        logger.error(f"Erreur lors de la suppression du document: {str(e)}")
        logger.error(traceback.format_exc())
        raise HTTPException(

```

Cette fonction définit un endpoint FastAPI accessible en méthode **DELETE** via l'URL

/documents/delete/, permettant à un utilisateur de supprimer un document précédemment téléversé et stocké dans la base de données MongoDB. Elle attend une requête JSON contenant deux champs : user_id (identifiant de l'utilisateur) et filename (nom du fichier à supprimer). Si l'un de ces deux champs est manquant, une erreur 400 est immédiatement renvoyée. Une fois les données validées, la fonction tente de supprimer un document correspondant à ce couple user_id / filename dans la collection docs_collection. Si aucun document n'est trouvé (suppression non effectuée), une erreur 404 est retournée pour informer que le fichier n'existe pas dans la base. En cas de succès, un message de confirmation est renvoyé. Toutes les opérations sont soigneusement journalisées à l'aide du système de logs, et en cas d'exception inattendue, l'erreur est enregistrée et une réponse HTTP 500 est émise pour indiquer une erreur interne du serveur.

```
if __name__ == "__main__":  
    import uvicorn  
    uvicorn.run(app, host="0.0.0.0", port=8000, reload=True)
```

Ce bloc de code permet de lancer l'application FastAPI de manière autonome en tant que script principal. L'instruction if __name__ == "__main__": signifie que ce code ne sera exécuté que si le fichier est lancé directement (et non importé comme module dans un autre fichier Python). À l'intérieur, uvicorn.run(...) démarre le serveur avec l'application FastAPI app sur l'adresse IP 0.0.0.0 (ce qui la rend accessible depuis n'importe quelle machine du réseau) et sur le port 8000. L'option reload=True permet de redémarrer automatiquement le serveur chaque fois qu'un changement est détecté dans le code source, ce qui est très pratique en phase de développement.

4. Fonctionnement des agents

1. DocumentRetrieverAgent

Rôle : Récupérer les documents les plus pertinents à partir de la question de l'utilisateur.

Fonctionnement :

- Il utilise l'outil DocumentRetrievalTool qui interroge la base vectorielle (Chroma) associée à l'utilisateur.
- Il effectue une recherche vectorielle (top k=8) en utilisant la similarité entre la question et les textes indexés.
- Ensuite, un re-ranker (CrossEncoder) trie les documents les plus pertinents selon une meilleure compréhension sémantique.
- Il retourne les 5 meilleurs documents au maximum.

Entrées : question, user_id

Sorties : retrieved_docs (liste de documents pertinents)

2. ContentAnalyzerAgent

Rôle : Analyser les documents récupérés pour produire un résumé structuré pertinent à la question.

Fonctionnement :

- Utilise l'outil ContentAnalysisTool, qui combine un prompt structuré avec un modèle de langage (llama3.2:1b).
- Le prompt guide le modèle à identifier :
 - Les informations clés pertinentes.
 - Les relations entre concepts.
 - Les éléments manquants.
- Un résumé synthétique.

Entrées : retrieved_docs, question

Sorties : analysis_result (texte structuré de l'analyse)

3. ResponseSynthesizerAgent

Rôle : Générer une réponse claire et précise à partir de l'analyse précédente.

Fonctionnement :

Le prompt utilisé est conditionné par la langue de la question (détection automatique via langdetect).

Il synthétise la réponse à partir de l'analyse (fournie par l'agent précédent) et la question initiale.

Utilise llama3.2:1b pour formuler une réponse fluide, concise et bien structurée dans la langue adéquate (français ou anglais).

Entrées : analysis_result, question

Sorties : synthesized_response (réponse finale rédigée)

4. QualityControllerAgent

Rôle : Évaluer la qualité de la réponse générée et éventuellement détecter les réponses peu fiables.

Fonctionnement :

Utilise un prompt d'évaluation qui demande au LLM d'attribuer une note entre 0 et 1.

La note est basée sur 4 critères :

- Pertinence
- Précision
- Complétude
- Clarté
- Si la note est inférieure à 0.6, un avertissement est logué (il est possible d'ajouter une logique de reformulation à ce stade).

Entrées : synthesized_response, question

Sorties : quality_score et final_response

4. Orchestrateur MultiAgentOrchestrator

Rôle : Gérer l'enchaînement de tous les agents comme un pipeline de traitement intelligent.

Fonctionnement :

Il orchestre les agents via un graph d'exécution avec 4 nœuds :

"retriever" → 2. "analyzer" → 3. "synthesizer" → 4. "quality_controller" → END

À chaque étape, l'état (AgentState) est enrichi et transmis à l'agent suivant.



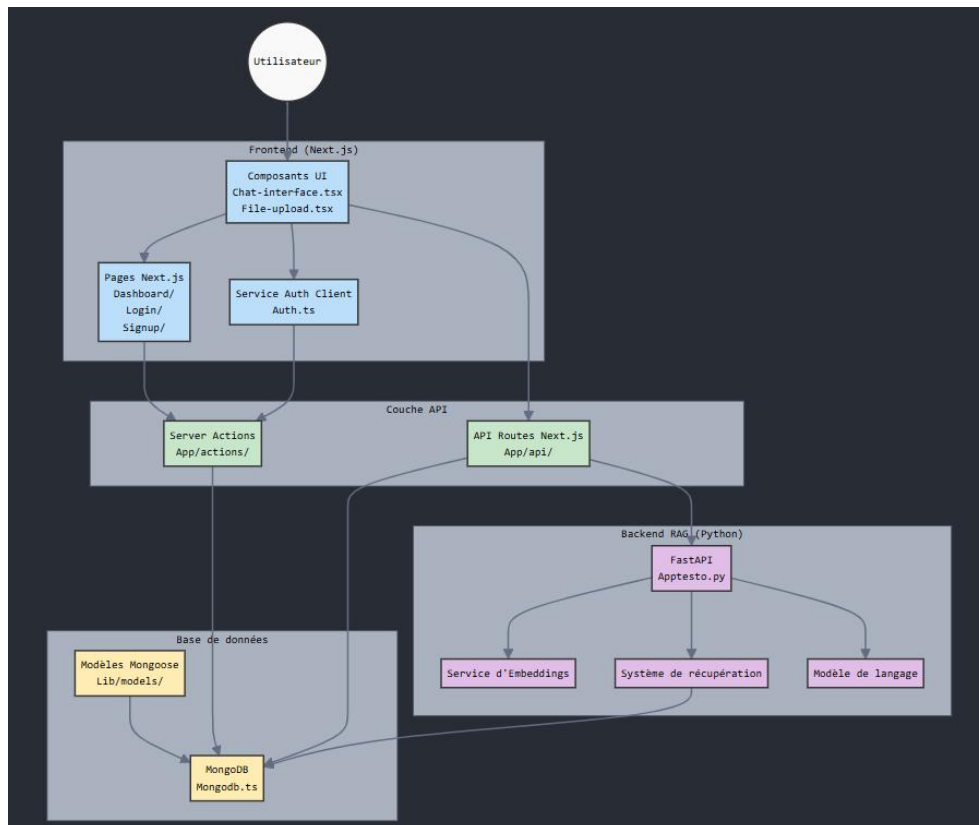
Conclusion du processus implémentation

Ce projet représente une API FastAPI robuste combinant deux systèmes : un pipeline vectoriel intelligent d'ingestion de documents et un système multi-agent raisonné pour le questionnement. Grâce à l'intégration de Chroma pour la recherche vectorielle, MongoDB pour la persistance et langchain pour l'orchestration des agents, l'architecture offre une réponse précise et contextualisée aux requêtes utilisateurs. L'usage des embeddings Ollama garantit une compréhension multilingue des contenus. En parallèle, un reranker basé sur CrossEncoder affine la pertinence des documents retrouvés. Les utilisateurs peuvent uploader, explorer ou supprimer leurs documents, avec support pour PDF, DOCX, TXT et extraction intelligente de tables. L'agent SMA orchestre quatre modules spécialisés : analyse, recherche, synthèse contextuelle et validation. Chaque réponse suit un parcours structuré, assurant robustesse et fiabilité. Le système assure également une gestion concurrente sécurisée via des Locks utilisateur. Tous les composants sont conçus pour être évolutifs, maintenables et facilement auditables. Cette architecture constitue une base solide pour des assistants documentaires intelligents.

III. Partie Frontend (Interface utilisateur)

1. Architecture générale du frontend

a. Diagramme des composants (vue structurelle)



Ce diagramme représente l'architecture structurelle de l'application, en montrant comment les différents modules sont organisés et interagissent entre eux.

Structure du diagramme de composants

- Frontend (Next.js)
 - Composants UI : Contient les interfaces utilisateur comme chat-interface.tsx et file-upload.tsx qui permettent l'interaction avec l'utilisateur
 - Pages Next.js : Les différentes pages de l'application (dashboard, login, signup)
 - Service Auth Client : Gère l'authentification côté client
- Couche API
 - API Routes Next.js : Points d'entrée HTTP dans app/api/
 - Server Actions : Fonctions serveur dans app/actions/ pour l'authentification et autres opérations
- Base de données
 - MongoDB : La base de données principale configurée via mongodb.ts
 - Modèles Mongoose : Définitions des structures de données dans lib/models/
- Backend RAG (Python)

- FastAPI : API Python (apptesto.py) qui expose les fonctionnalités RAG
- Service d'Embeddings : Transforme les documents en représentations vectorielles
- Système de récupération : Recherche les documents pertinents pour une requête
- Modèle de langage : Génère les réponses basées sur le contexte et la question

Relations entre les composants

- L'Utilisateur interagit directement avec les Composants UI
- Les Composants UI communiquent avec:
 - Les Pages pour la navigation
 - Le Service Auth Client pour l'authentification
 - Les API Routes pour les requêtes de données
- Les Pages et le Service Auth Client utilisent les Server Actions
- Les Server Actions et API Routes accèdent à MongoDB
- Les API Routes communiquent avec FastAPI pour les fonctionnalités RAG
- FastAPI orchestre les trois composants du système RAG:
 - Le Service d'Embeddings
 - Le Système de récupération
 - Le Modèle de langage
- Le Système de récupération accède à MongoDB pour chercher les documents pertinents
- Les Modèles Mongoose définissent la structure des données dans MongoDB

b. Diagramme de séquence (vue dynamique)



Ce diagramme illustre les interactions chronologiques entre les différents composants de votre système, en montrant comment ils communiquent entre eux pour réaliser diverses fonctionnalités. Voici l'explication des quatre principales séquences représentées :

Processus d'authentification

- L'utilisateur (U) se connecte via l'interface frontend (FE)
- Le frontend envoie les identifiants à l'API
- L'API vérifie ces identifiants dans la base de données MongoDB (DB)
- La DB confirme la validité des identifiants
- L'API renvoie une session authentifiée au frontend

Téléchargement et traitement de documents

- L'utilisateur télécharge un document via l'interface

- Le frontend transmet ce document à l'API
- L'API stocke le document dans la base de données
- L'API envoie également le document au backend RAG (FastAPI)
- Le backend RAG génère des embeddings (représentations vectorielles) du document
- Le RAG confirme le traitement à l'API
- L'API confirme au frontend que le document a été téléchargé et traité

Processus de question-réponse (cœur du système RAG)

- L'utilisateur pose une question
- Le frontend transmet cette question à l'API
- L'API envoie la question au système RAG
- Le RAG interroge la base de données pour récupérer les documents pertinents à la question
- La DB retourne le contexte documentaire approprié
- Le RAG combine la question et le contexte pour les envoyer au modèle de langage (LLM)
- Le LLM génère une réponse basée sur la question et le contexte
- Cette réponse est transmise au RAG, qui la renvoie à l'API
- L'API transmet la réponse au frontend
- Le frontend présente finalement la réponse à l'utilisateur

Gestion des paramètres utilisateur

- L'utilisateur modifie ses paramètres/préférences
- Le frontend envoie ces modifications à l'API
- L'API met à jour le profil utilisateur dans la base de données
- La DB confirme la mise à jour
- L'API informe le frontend que les paramètres ont été appliqués

2. Les composants de l'interface

a. Page d'accueil



b. Page d'inscription

← Retour à l'accueil

Créer un compte

Inscrivez-vous pour commencer à utiliser votre assistant RAG

Prénom

Nom

Email

Mot de passe

Confirmer le mot de passe

S'inscrire

[Vous avez déjà un compte? Se connecter](#)

La page d'inscription a pour objectif de permettre aux nouveaux utilisateurs de créer un compte personnel afin d'accéder à l'assistant RAG via un formulaire sécurisé et ergonomique. La page principale côté serveur et a pour rôle de gérer l'accès à cette page d'inscription. Lorsqu'un utilisateur est déjà connecté il est automatiquement redirigé vers le tableau de bord .

Dans le cas contraire, **la page d'inscription** s'affiche et comprend un lien de retour vers la page d'accueil, ainsi qu'un encadré contenant un titre, une description, et un lien de redirection vers la page de connexion pour les utilisateurs disposant déjà d'un compte.

c. Page de connexion

← Retour à l'accueil

Connexion

Connectez-vous pour accéder à votre assistant RAG

Email

Mot de passe

Se connecter

[Vous n'avez pas de compte? S'inscrire](#)

La page de connexion permet aux utilisateurs déjà enregistrés de se connecter à leur compte afin d'accéder à leur espace personnel sur l'assistant RAG.

Le rôle de la page principale côté serveur est de vérifier si l'utilisateur dispose déjà d'une session active. Si c'est le cas, il est automatiquement redirigé vers le tableau de bord, ce qui empêche tout retour inutile à la page de connexion.

Dans le cas contraire, une interface claire et minimaliste est affichée. Celle-ci comprend un lien vers la page d'accueil ainsi qu'un encadré contenant le titre "Connexion", le sous-titre "Connectez-vous pour accéder à notre assistant RAG", le formulaire LoginForm, et un lien vers la page d'inscription destiné aux nouveaux utilisateurs.

d. Page en savoir plus

L'objectif de cette page est de détailler comment fonctionne la technologie RAG (Retrieval-Augmented Generation) intégrée à notre application. Elle explique comment cette méthode fusionne l'intelligence des modèles de langage et l'examen de documents spécifiques pour fournir des réponses plus adaptées et contextuelles.

Comprendre notre système RAG

Découvrez comment notre technologie de Retrieval-Augmented Generation (RAG) révolutionne l'interaction avec vos documents et données.

Qu'est-ce que RAG ?

RAG (Retrieval-Augmented Generation) est une approche avancée qui combine la puissance des grands modèles de langage (LLM) avec une recherche précise dans vos propres documents.

Contrairement aux chatbots traditionnels qui s'appuient uniquement sur leurs connaissances préentraînées, notre système RAG peut rechercher des informations spécifiques dans vos documents et les utiliser pour générer des réponses précises et contextuelles.

Cette approche hybride permet d'obtenir des réponses plus pertinentes, plus précises et toujours à jour avec vos informations les plus récentes.

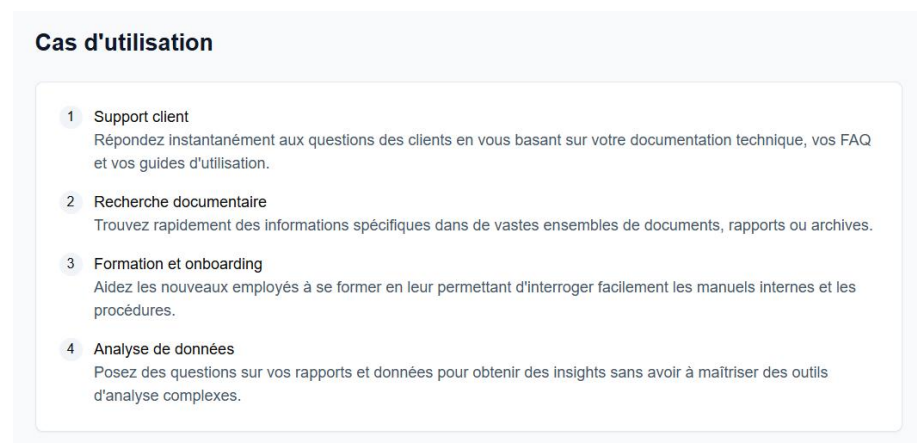
Retour à l'accueil : Un lien pour revenir à la page principale de l'application.

Présentation de RAG : Un aperçu de ce qu'est le système RAG et comment il fonctionne, en contrastant avec les chatbots traditionnels.



Fonctionnement du système RAG : Cette section décrit les étapes clés du système RAG

Avantages du système RAG : Présentation des bénéfices du système RAG



Cas d'utilisation : Exemples de situations où RAG peut être utile



Questions fréquentes (FAQ) : Réponses à des questions courantes.

Nous contacter

Pour toute question, demande d'information ou assistance, vous pouvez nous contacter aux adresses suivantes :

- ☐ Rabab HANZAZ – hanzazrabab4@gmail.com
- ☐ Yasmina El HAFI – Yasminaelhafi2@gmail.com
- ☐ Oumaima LOUALI – Louailoumaima13@gmail.com

Commencer maintenant

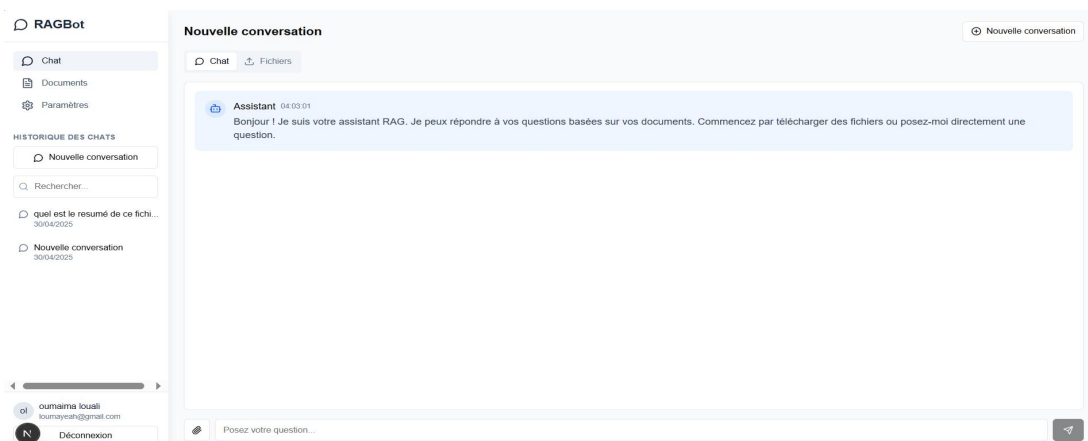
© 2025 RAGBot. Tous droits réservés.

Contact : Informations de contact des responsables .

Appel à l'action : Un bouton pour inviter les utilisateurs à s'inscrire et commencer à utiliser l'application.

e. Page dashboard

Page de chat



Cette page définit la page principale du tableau de bord de l'application, intégrant une barre latérale réactive pour naviguer entre les sections (Chat, Documents, Paramètres), tout en gérant l'affichage des informations utilisateur, l'historique des conversations, et la déconnexion. Il offre également un bouton pour afficher/masquer la barre latérale en version mobile, et permet de charger dynamiquement le contenu principal selon la section sélectionné.

Historique de chat :

Ce composant affiche l'historique des conversations dans le tableau de bord, tout en offrant des fonctionnalités de recherche et de suppression. Il utilise des données simulées contenant un identifiant, un titre, un aperçu et une date pour chaque conversation. L'utilisateur peut rechercher dynamiquement une conversation par titre, et les dates sont affichées sous un format lisible (ex. « Aujourd'hui », « Hier »). Enfin, chaque conversation peut être supprimée grâce à un bouton qui s'affiche au

survol.

Page de téléchargement d'un fichier

The screenshot shows a web interface titled 'Conversation' with a 'Nouvelle conversation' button. Below the title bar, there are tabs for 'Chat' and 'Fichiers'. The main section is titled 'Document de référence' with a subtitle 'Le document sera utilisé comme source de connaissances pour le chatbot'. It features a large dashed box for file upload, containing an upload icon and the text 'Glissez-déposez votre fichier ici ou Sélectionner un fichier'. Below the box, it lists 'Formats supportés : PDF, DOCX, TXT (max 10MB)'. At the bottom, a note states: 'Le document téléchargé sera utilisé comme source de connaissances pour répondre aux questions.'

Ce composant permet à l'utilisateur de téléverser des fichiers (PDF, DOCX, TXT...). Il propose une zone de glisser-déposer ou de sélection manuelle, affiche les fichiers ajoutés avec leurs détails (nom, taille, statut) et gère le suivi du téléchargement avec barre de progression, icônes de succès ou d'erreur, et option de suppression. L'interface est interactive, claire, et adaptée à différents formats de fichiers dans des limites de taille prédéfinies.

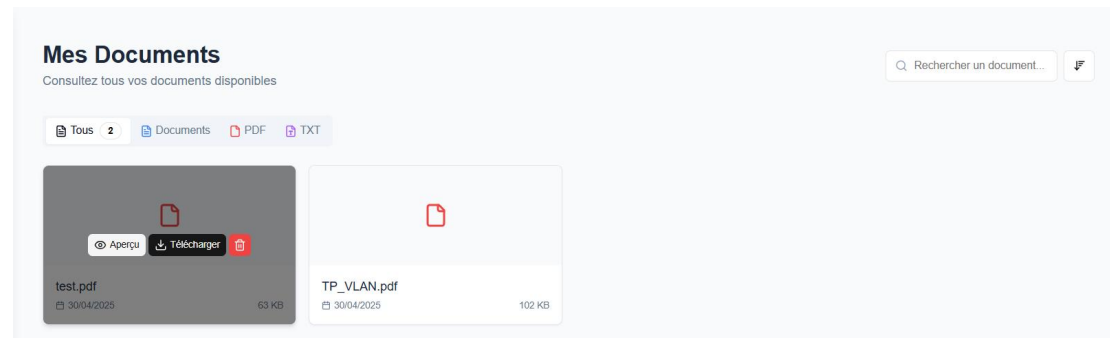
Page de paramètres

The screenshot shows a 'Paramètres' (Settings) page with two main sections. The first section, 'Profil utilisateur', contains input fields for 'Prénom', 'Nom', and 'Email', followed by an 'Enregistrer les modifications' button. The second section, 'Sécurité', contains input fields for 'Mot de passe actuel', 'Nouveau mot de passe', and 'Confirmer le mot de passe', followed by a 'Changer le mot de passe' button.

Cette page, intégrée au layout du dashboard pour assurer une interface cohérente, est structurée en trois sections : un espace de profil utilisateur pour modifier prénom,

nom et email avec enregistrement des changements et un espace sécurité dédié au changement de mot de passe via un formulaire sécurisé.

Page d'archive des documents



Cette interface permet à l'utilisateur de gérer ses documents. Elle affiche un titre principal "Mes Documents" avec un sous-texte invitant à consulter les fichiers disponibles. En haut, des boutons de filtrage permettent de trier les documents par type (Tous, Documents, PDF, TXT), tandis qu'un champ de recherche et un bouton de tri facilitent la navigation. Les documents sont présentés sous forme de cartes contenant le nom du fichier, sa date d'ajout, sa taille, et une icône représentant son format. Lorsqu'on survole une carte, des actions apparaissent : aperçu, téléchargement, ou suppression du fichier.

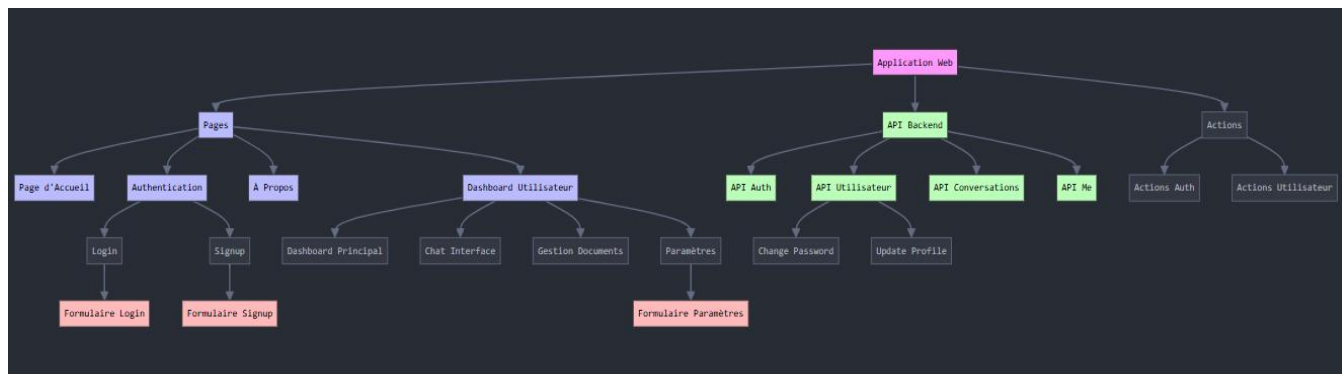
3. Architecture détaillée des composants

a. Racine : app/

C'est le dossier central dans le modèle App Router de Next.js. Chaque sous-dossier représente une route.

app/	
├─ about/	← Page "À propos" de l'application
├─ actions/	
│ └─ auth.ts	← Fonctions liées à l'authentification
│ └─ user.ts	← Fonctions liées à la gestion des utilisateurs
├─ api/	← Endpoints API pour le backend
│ └─ auth/	← Authentification côté API
│ └─ conversations/	
│ └─ [id]/	← Récupération d'une conversation par ID
│ └─ me/	
│ └─ route.ts	← Endpoint pour récupérer les infos de l'utilisateur connecté
│ └─ user/	
│ └─ change-password/	← API pour changer le mot de passe
│ └─ update-profile/	← API pour mettre à jour le profil utilisateur

└─ dashboard/	← Espace utilisateur connecté
└─ chat/	
└─ [id]/	
└─ page.tsx	← Interface de discussion avec un chatbot par ID
└─ documents/	
└─ loading.tsx	← Composant de chargement pour les documents
└─ page.tsx	← Interface de gestion des documents
└─ settings/	
└─ page.tsx	← Page de paramètres du dashboard
└─ settings-form.tsx	← Formulaire de mise à jour des paramètres
└─ page.tsx	← Page principale du dashboard
└─ login/	
└─ login-form.tsx	← Formulaire de connexion
└─ page.tsx	← Page de connexion
└─ signup/	
└─ signup-form.tsx	← Formulaire d'inscription
└─ page.tsx	← Page d'inscription
└─ globals.css	← Feuille de style globale de l'application
└─ layout.tsx	← Mise en page globale de l'application
└─ page.tsx	← Page d'accueil principale (landing page)



◆ **auth, user, login, signup**

Ces dossiers sont liés à l'authentification et à la gestion des utilisateurs :

- auth.ts et user.ts dans actions/ contiennent probablement des fonctions serveur (server actions).
- login/, signup/ contiennent des composants comme login-form.tsx, signup-form.tsx : formulaires de connexion et d'inscription.
- change-password, update-profile : des routes pour gérer les modifications de compte utilisateur.

◆ **api/**

Ce dossier suit la convention de l'App Router pour définir des routes API côté serveur :

- auth, user, me, conversations : définissent les endpoints pour l'authentification, les utilisateurs, les informations personnelles et les conversations.
- route.ts : chaque fichier contient une fonction (probablement un handler GET/POST/etc.).

■ **dashboard/**

C'est le backoffice ou l'espace personnel de l'utilisateur connecté :

- chat/[id]/page.tsx : une route dynamique pour des conversations individuelles.
- documents/page.tsx et loading.tsx : chargement de documents, peut-être avec un état de chargement asynchrone.
- settings/ : configuration utilisateur avec settings-form.tsx.

■ **Fichiers globaux**

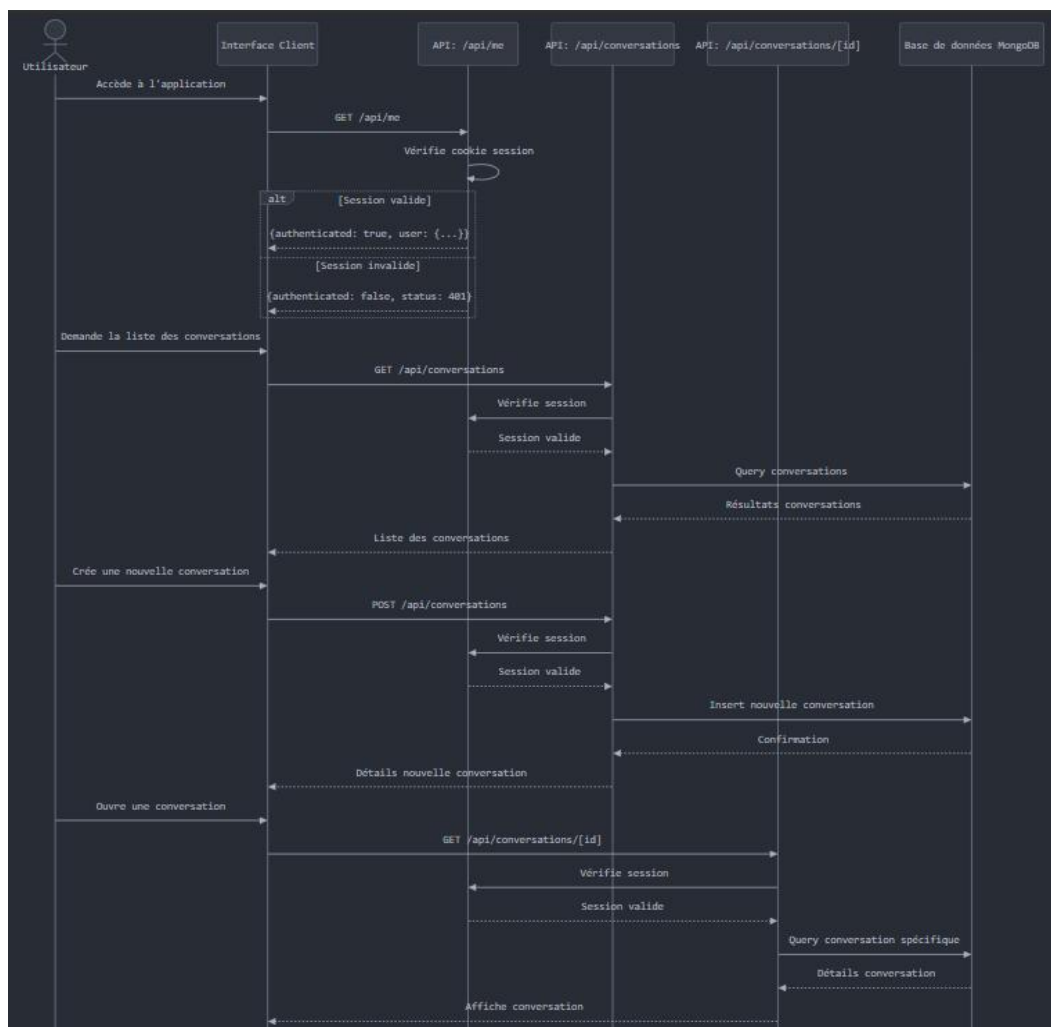
- globals.css : styles CSS globaux.
- layout.tsx : layout général (entêtes, barres latérales, etc.).
- page.tsx (à la racine de app/) : page d'accueil ou route par défaut.

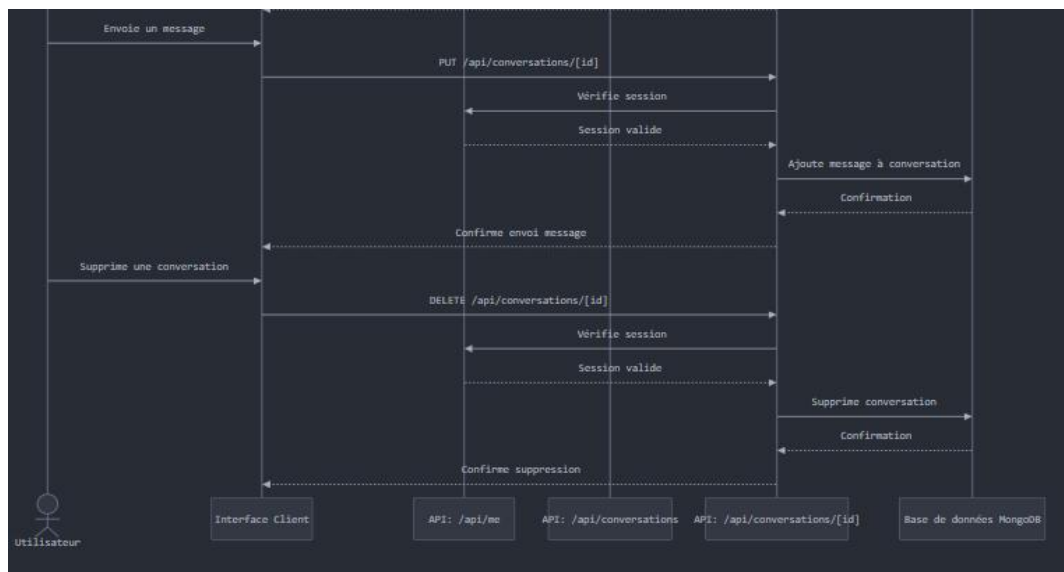
i. **Action d'authentification et gestion utilisateur app/actions**



Ce diagramme démontre les interactions entre les différentes couches de l'application lors des opérations d'authentification des utilisateurs et de gestion de profil. L'architecture moderne implique que le client interagisse avec des pages d'interface, qui sont en communication avec les actions maîtres de la logique côté serveur. Ces actions font appel soit à la Base de Données, soit transitent par l'API Backend en fonction de l'opération. Concernant l'inscription et la connexion, les actions interagissent directement avec la base de données pour contrôler et enregistrer les informations de l'utilisateur avant de créer un cookie de session sécurisé. La déconnexion est simple et nécessite la suppression d'un cookie de session ainsi qu'une redirection. L'API Backend joue le rôle d'intermédiaire avec la Base de Données, garantissant une distinction claire des responsabilités et renforçant ainsi la sécurité des données utilisateur.

ii. Gestion des conversations





1. /api/me/route.ts

Ce fichier contient une simple route GET qui vérifie l'authentification de l'utilisateur, il assure que seuls les utilisateurs authentifiés peuvent accéder aux fonctionnalités:

- Accède aux cookies de la requête
- Vérifie si un cookie "session" existe
- Si présent, extrait les informations utilisateur et les retourne
- Sinon, retourne un état non authentifié (401)

Cette route sert essentiellement à confirmer si un utilisateur est connecté et à récupérer ses informations de base.

2. /api/conversations/[id]/route.ts

Ce fichier gère les opérations CRUD sur une conversation spécifique, Permet de consulter, modifier (ajouter des messages) et supprimer des conversations spécifiques via trois méthodes HTTP :

- **GET** : Récupère les détails d'une conversation
 - Vérifie l'authentification via getSession()
 - Extrait l'ID de la conversation des paramètres d'URL
 - Interroge la base MongoDB pour trouver la conversation associée à cet ID et à l'utilisateur actuel
 - Retourne la conversation ou une erreur 404 si non trouvée
- **PUT** : Ajoute un nouveau message à une conversation existante
 - Vérifie l'authentification
 - Extrait l'ID de conversation et le message du corps de la requête
 - Trouve la conversation appropriée et y ajoute le nouveau message
 - Sauvegarde les modifications et confirme le succès de l'opération

- **DELETE** : Supprime entièrement une conversation
 - Vérifie l'authentification
 - Supprime la conversation correspondant à l'ID et à l'utilisateur actuel
 - Confirme la suppression ou retourne une erreur 404 si non trouvée

3. /api/conversations/route.ts

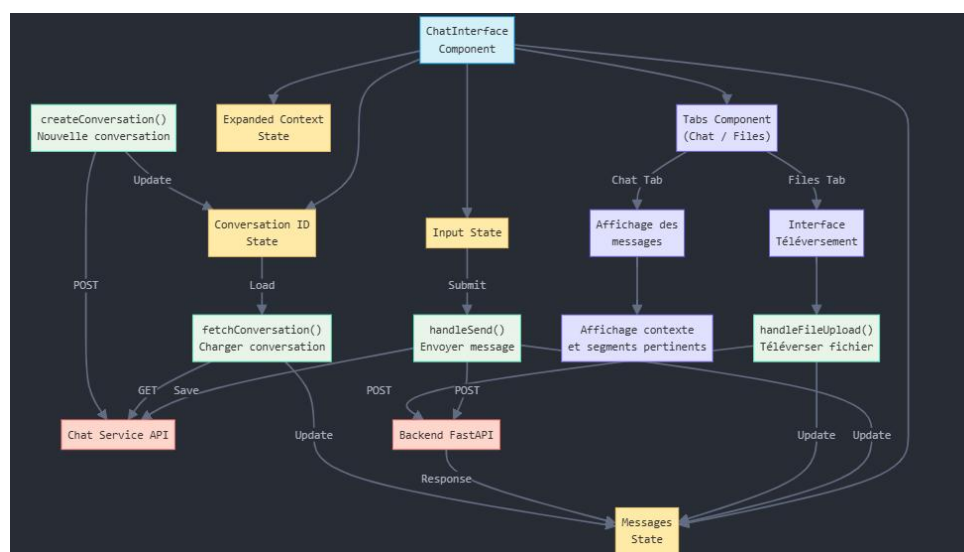
Ce fichier gère les opérations globales sur les conversations ,permet de lister toutes les conversations existantes et d'en créer de nouvelles avec un message d'accueil automatique:

- **GET** : Liste toutes les conversations de l'utilisateur
 - Vérifie l'authentification
 - Récupère toutes les conversations appartenant à l'utilisateur actuel
 - Trie les résultats par date de mise à jour (descendant)
 - Sélectionne uniquement les champs nécessaires pour l'affichage
- **POST** : Crée une nouvelle conversation
 - Vérifie l'authentification
 - Crée une nouvelle conversation dans la base de données avec :
 - L'ID de l'utilisateur actuel
 - Un titre par défaut "Nouvelle conversation"
 - Un message initial de bienvenue de l'assistant RAG
 - Retourne les informations de base de la nouvelle conversation

b. /components

components/	# Composants réutilisables de l'interface utilisateur
├─ dashboard/	# Composants spécifiques au tableau de bord
│ └─ chat-history.tsx	# Gestion de l'historique des conversations
│ └─ chat-interface.tsx	# Interface de chat principale
│ └─ file-upload.tsx	# Composant de téléchargement de fichiers
│ └─ layout.tsx	# Structure de mise en page du tableau de bord
│ └─ theme-provider.tsx	# Gestion du thème de l'application
└─ ui/	# Composants d'interface utilisateur génériques
└─ test	# Tests des composants
data/	# Gestion des données et états
hooks/	# Hooks React personnalisés
├─ use-mobile.tsx	# Hook pour la gestion des appareils mobiles
└─ use-toast.ts	# Hook pour les notifications toast
lib/	# Bibliothèques et utilitaires
├─ models/	# Modèles de données
├─ api-service.ts	# Services pour les appels API
├─ auth.ts	# Gestion de l'authentification
├─ mongodb.ts	# Configuration et utilitaires MongoDB
└─ utils.ts	# Fonctions utilitaires diverses
node_modules/	# Dépendances externes (packages npm)

i. L'interface de chat



le composant principal ChatInterface orchestre l'interaction entre différents états et fonctionnalités. Au cœur du système, les états de Messages, d'Input et de ConversationID gèrent les données, tandis que les fonctions handleSend() et handleFileUpload() servent de ponts vers les services API externes.

L'interface utilisateur est structurée autour d'un système d'onglets permettant de basculer entre la conversation et la gestion des fichiers. Lorsqu'un utilisateur envoie un message, le flux de données traverse handleSend(), communique avec le backend FastAPI pour obtenir une réponse, puis met à jour l'état des messages pour affichage. Ce système permet également l'affichage contextuel des informations pertinentes

extraites des documents téléversés, créant ainsi une expérience utilisateur intégrée où conversations et données documentaires coexistent de manière fluide

Cette interface permet aux utilisateurs d'interagir avec un assistant IA capable de répondre à des questions en s'appuyant sur les documents téléchargés par les utilisateurs.

Structure du composant principal

L'application est organisée autour d'un élément central, ChatInterface, qui prend en charge l'ensemble de l'expérience utilisateur. Ce composant conserve divers états cruciaux, comme la liste des messages, le texte que l'utilisateur a entré, l'identifiant de la conversation actuelle ainsi que différents états d'affichage pour situer les réponses. Il fait un usage intensif des hooks React (useState, useRef, useEffect) pour contrôler ces états et provoquer les effets secondaires adéquats.

Flux de travail de l'application

L'application offre un processus de travail intuitif : l'utilisateur a la possibilité d'initier une nouvelle conversation ou d'ouvrir une précédente, de poser des questions à l'assistant ou d'importer des fichiers pour compléter le répertoire de connaissances. Quand une interrogation est formulée, celle-ci est transmise à l'API de backend qui examine les documents à sa disposition afin de produire une réponse appropriée. Cette réponse est par la suite présentée sur l'interface avec le contexte utilisé pour sa création, ce qui permet à l'utilisateur de saisir la provenance de l'information.

Gestion avancée du contexte

Un aspect particulièrement captivant de cette interface est la gestion sophistiquée du contexte employé. Pour chaque réponse fournie par l'assistant, l'utilisateur a la possibilité d'examiner les extraits des documents ayant contribué à la formulation de ladite réponse. Ces extraits sont organisés en fonction de leur pertinence et présentés avec les mots-clés de la question soulignés. L'utilisateur a la possibilité de changer entre diverses perspectives, telles que l'affichage uniquement des segments les plus significatifs ou la visualisation des extraits dans leur intégralité.

Système de gestion des fichiers

L'interface offre également un système de gestion de fichiers avec une limite de trois documents par utilisateur. Les fichiers téléchargés sont traités par le backend et confirmés par des messages dans l'interface. Cette fonctionnalité est essentielle pour alimenter la base de connaissances utilisée par l'assistant.

Persistance des conversations

Un service dédié assure la conservation des conversations en stockant les messages sur le serveur. L'identifiant de la conversation actuelle est aussi conservé dans le localStorage du navigateur afin de faciliter la reprise d'une discussion en cours pour l'utilisateur.

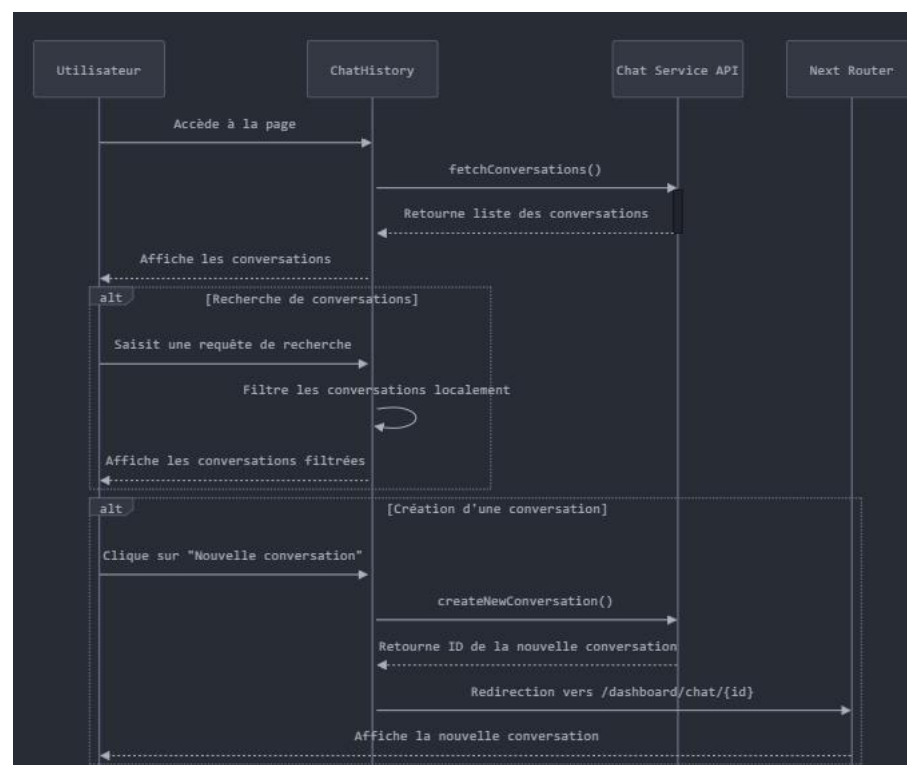
Interface utilisateur et organisation

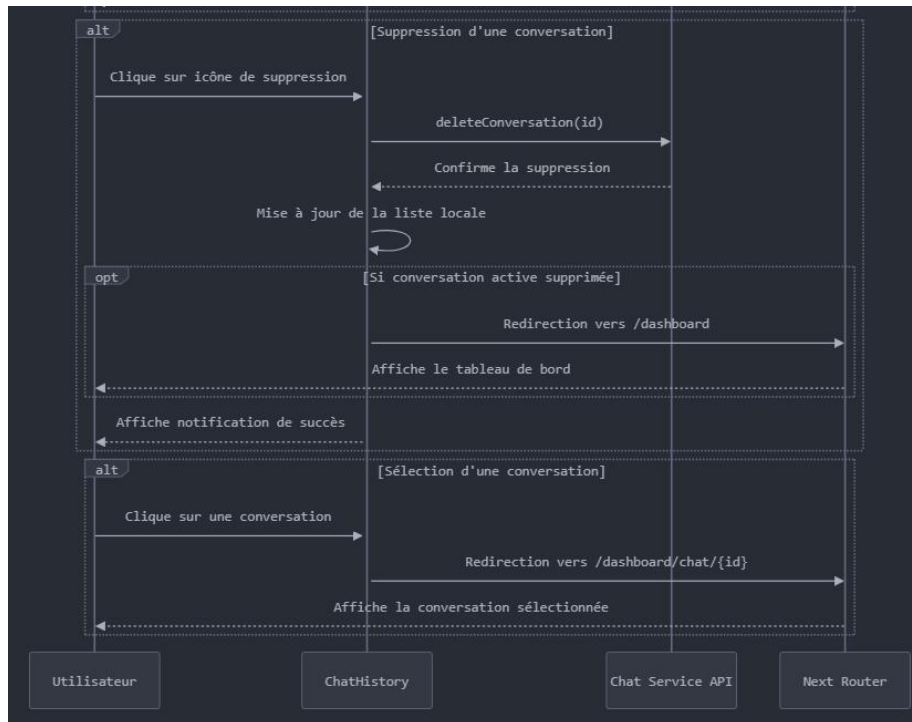
L'interface utilisateur est structurée en onglets qui facilitent la transition entre la vue principale de discussion et la gestion des fichiers. L'espace de discussion présente l'historique des messages, en différenciant visuellement les interventions de l'utilisateur et celles de l'assistant. La partie de saisie comporte un champ de texte, un bouton pour soumettre et un bouton pour le téléchargement de fichiers.

Expérience utilisateur et explicabilité

L'application met l'accent sur l'expérience utilisateur en intégrant des barres de progression, des messages d'erreur explicites et un design réactif qui s'ajuste aux diverses dimensions d'écran. Des caractéristiques explicatives telles que la présentation du contexte et l'accentuation des termes clés aident à rendre le système d'intelligence artificielle plus clair et intelligible pour l'utilisateur.

ii. L'historique de chat





Le fichier chat-history.tsx est un composant React côté client qui gère l'affichage et l'interaction avec l'historique des conversations d'un utilisateur. Ce composant fait partie d'une interface de chat plus large et permet de visualiser, rechercher, créer et supprimer des conversations.

Structure des données

Le fichier définit une interface Conversation qui structure les données de chaque conversation avec quatre propriétés essentielles : id (identifiant unique), title (titre de la conversation), createdAt (date de création) et updatedAt (date de dernière mise à jour). Cette structure assure une manipulation cohérente des données à travers le composant.

Gestion de l'état

Le composant utilise plusieurs hooks d'état React pour gérer différentes données :

- **conversations** : stocke la liste des conversations récupérées depuis l'API
- **searchQuery** : contient la requête de recherche saisie par l'utilisateur
- **isLoading** : indique si une requête de chargement est en cours Ces états permettent au composant de réagir dynamiquement aux interactions de l'utilisateur et aux changements de données.

Chargement des données

La fonction `fetchConversations()` est appelée au montage du composant et à chaque changement de chemin d'URL. Elle interroge l'API via `chatService.getConversations()`, puis transforme les données reçues en un format utilisable par le composant. Cette fonction gère également les états de chargement et les erreurs potentielles à l'aide du système de notifications toast.

Gestion des conversations

Le composant implémente trois fonctionnalités principales pour la gestion des conversations :

- **Suppression** : La fonction `deleteConversation()` permet de supprimer une conversation spécifique en appelant `chatService.deleteConversation()`, puis en mettant à jour l'état local des conversations et en redirigeant l'utilisateur si nécessaire.
- **Création** : La fonction `createNewConversation()` crée une nouvelle conversation via l'API et redirige l'utilisateur vers celle-ci.
- **Filtrage** : La variable `filteredConversations` applique la requête de recherche aux conversations existantes pour afficher uniquement les résultats pertinents.

Fonctions utilitaires

Le composant utilise des fonctions auxiliaires pour améliorer l'expérience utilisateur : `formatDate()` : transforme les dates en format lisible et contextuel (comme "Aujourd'hui", "Hier", etc.)

`isConversationActive()` : détermine si une conversation est actuellement active en comparant son ID avec l'URL courante

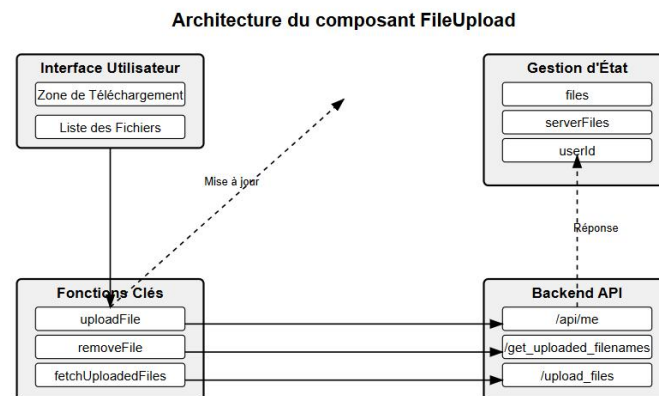
Gestion de la navigation

Le composant utilise le hook `useRouter` de Next.js pour gérer la navigation entre les différentes conversations. Lorsqu'un utilisateur clique sur une conversation, le router redirige vers `/dashboard/chat/[id]` où l'ID correspond à celui de la conversation sélectionnée.

Gestion des erreurs

Le système implémente une gestion des erreurs robuste à travers l'utilisation du hook `useToast` qui affiche des notifications contextuelles en cas d'échec des opérations API, offrant ainsi un retour d'information clair à l'utilisateur sur l'état de ses actions.

iii. Téléversement des fichiers



Le fichier `file-upload.tsx` est un composant React construit avec Next.js qui permet aux utilisateurs de télécharger des documents de référence pour un chatbot. Il offre une interface utilisateur intuitive pour le glisser-déposer ou la sélection de fichiers, et gère l'intégralité du processus de téléchargement, d'affichage et de suppression des fichiers.

Structure des données

Le composant utilise deux structures de données principales : `UploadedFile` pour suivre l'état des fichiers côté client et `FileInfo` pour représenter les fichiers stockés sur le serveur. Ces interfaces permettent de suivre l'état de chaque fichier, y compris son nom, sa taille, son type, sa progression de téléchargement et son statut (en cours, terminé ou erreur).

Gestion de l'état

Le composant utilise plusieurs états React pour gérer différentes parties de son fonctionnement, notamment :

- `files` pour stocker les fichiers téléchargés
- `serverFiles` pour les fichiers récupérés du serveur
- `isDragging` pour l'état du glisser-déposer
- `userId` pour l'identifiant de l'utilisateur actuel
- `errorMessage` pour afficher les messages d'erreur
- `isUploading` et `isLoading` pour les indicateurs de chargement

Cette structure permet au composant de réagir efficacement aux actions de l'utilisateur et aux réponses du serveur.

Interactions avec l'API

Le composant interagit avec une API backend via plusieurs points d'accès :

- `/api/me` pour récupérer l'ID de l'utilisateur
- `http://127.0.0.1:8000/get_uploaded_filenames/{userId}` pour récupérer les fichiers existants
- `http://127.0.0.1:8000/upload_files/` pour télécharger de nouveaux fichiers
- `http://127.0.0.1:8000/delete_file_vector/` pour supprimer des fichiers

Ces interactions sont gérées de façon asynchrone avec des états de chargement appropriés pour maintenir une bonne expérience utilisateur.

Fonctionnalités principales

Le composant offre plusieurs fonctionnalités clés :

- Téléchargement de fichiers par glisser-déposer ou sélection via un explorateur de fichiers
- Affichage de la progression du téléchargement avec une barre visuelle
- Limitation du nombre de fichiers (maximum 3) et des types de fichiers (PDF, DOCX, TXT)
- Affichage des fichiers existants avec leur statut, taille et type
- Possibilité de supprimer des fichiers individuels
- Actualisation de la liste des fichiers

Gestion des erreurs

Le système intègre une gestion robuste des erreurs à plusieurs niveaux :

- Validation des types de fichiers et de leur taille avant le téléchargement
- Affichage des erreurs spécifiques à chaque fichier
- Message d'erreur global pour les problèmes généraux
- Gestion des réponses d'erreur de l'API
- Cette approche assure que l'utilisateur est toujours informé de l'état de ses fichiers et des problèmes éventuels.

Optimisations et fonctionnalités avancées

Le composant inclut plusieurs optimisations :

- Simulation de la progression du téléchargement pour un retour visuel immédiat
- Gestion de la conversion des tailles de fichiers pour un affichage convivial
- Détection automatique des types de fichiers à partir de leur extension
- Rafraîchissement automatique de la liste après les opérations

4. Outils et technologies utilisés

- **TypeScript:** TypeScript est une surcouche de JavaScript qui introduit un système de typage statique. Cela permet de détecter les erreurs dès l'écriture du code, d'améliorer l'autocomplétion et de rendre le projet plus fiable, maintenable et évolutif. Il est particulièrement utile pour les projets complexes où la clarté et la robustesse du code sont essentielles.
- **Shadcn/ui:** Shadcn/ui est une bibliothèque de composants UI réutilisables et accessibles basée sur Radix UI et stylisée avec Tailwind CSS. Elle fournit des composants comme des boutons, des formulaires, des modales, etc., prêts à l'emploi tout en étant facilement personnalisables. Elle encourage les bonnes pratiques en matière d'accessibilité et d'expérience utilisateur.
- **Tailwind CSS:** Tailwind CSS est un framework utilitaire de styles CSS qui permet de construire des interfaces rapidement en appliquant directement des classes aux éléments HTML. Il offre un contrôle précis sur le design, une excellente réactivité, et une personnalisation facile via un fichier de configuration. C'est

idéal pour construire des interfaces modernes et cohérentes sans écrire beaucoup de CSS personnalisé.

- **Lucide React:** Lucide React est une bibliothèque d'icônes SVG modernes conçue pour React. Elle propose un large éventail d'icônes esthétiques, cohérentes, et personnalisables via Tailwind ou des props React. Elle est légère et parfaite pour des interfaces élégantes et fonctionnelles.

Conclusion du chapitre

Dans ce chapitre, nous avons approfondi les implications pratiques de l'intégration de la recherche intelligente (RAG) dans les modèles de langage. Voici les points clés à retenir :

Importance de la RAG : Nous avons vu que la RAG joue un rôle crucial en permettant aux modèles de langage de rester pertinents et informés, même lorsqu'ils sont confrontés à des questions complexes ou à des informations récentes. Cela est particulièrement pertinent dans un monde où les données évoluent rapidement .

Limites des modèles de langage : Bien que des modèles comme Llama 3 soient puissants, ils ont des limitations, notamment en ce qui concerne la longueur du contexte qu'ils peuvent traiter. Cela signifie qu'ils ne peuvent pas toujours intégrer toutes les informations nécessaires pour répondre de manière précise .

Outils et technologies : L'utilisation d'outils comme LangChain et Ollama a été mise en avant comme essentielle pour améliorer la compréhension des chatbots. Ces outils permettent d'accéder à des données à jour, ce qui enrichit les réponses fournies par les modèles de langage .

Conservation des conversations : Nous avons également discuté de l'importance de la persistance des conversations, qui permet aux utilisateurs de reprendre facilement leurs discussions. Cela est réalisé grâce à un stockage efficace des messages et des identifiants de conversation .

Prochaines étapes : En conclusion, ce chapitre a posé les bases pour la construction d'un système intégré qui utilise la RAG pour améliorer l'interaction avec les utilisateurs. Dans le prochain chapitre, nous allons explorer les étapes concrètes de la mise en œuvre de ce système, en détaillant les processus nécessaires pour créer une expérience utilisateur fluide et efficace.

Chapitre 3 - Comparaison entre le système RAG proposé (RAGbot) et les autres projets existants (SciSpace Copilot)

I. Introduction à SciSpace Copilot

SciSpace (anciennement connu sous le nom de Typeset) est une plateforme innovante qui vise à simplifier la lecture et la compréhension d'articles scientifiques complexes. Son produit phare, SciSpace Copilot, est un assistant de lecture intelligent basé sur l'intelligence artificielle.

1. Fonctionnement technique (RAG simplifié)

- **Extraction du contenu PDF**

Utilisation d'un moteur de parsing comme PyMuPDF pour extraire le texte, les figures, équations, etc.

- **Prétraitement et chunking**

Division du document en blocs de texte sémantiquement cohérents (paragraphe, titres, légendes...).

- **Vectorisation (Embeddings)**

Application de modèles d'embeddings sémantiques (OpenAI embeddings, ou modèles Hugging Face) pour convertir chaque bloc en vecteur numérique.

- **Indexation avec FAISS**

Stockage des vecteurs dans une base de données vectorielle (FAISS), permettant une recherche rapide par similarité.

- **Récupération (Retrieval)**

Lorsqu'une question est posée, les blocs les plus pertinents sont récupérés en

fonction de leur proximité vectorielle avec la requête.

- **Génération (avec un LLM)**

Ces blocs sont utilisés comme contexte pour générer une réponse via un modèle de langage comme GPT-3.5/4.

2. Fonctionnalités principales

- **Chat avec des PDF scientifiques**

Posez des questions sur des sections spécifiques, des figures ou des équations.

- **Explications de contenu complexe**

Résumés, simplification de jargon scientifique.

- **Recherche intelligente dans le document**

Rechercher rapidement un concept, un terme ou une citation.

- **Intégration avec Zotero**

Interaction avec votre bibliothèque scientifique personnelle.

- **Extensions de navigateur et Apps mobiles**

Accès à l'outil depuis Chrome, Android et iOS.

II. Comparaison

1. Architecture Générale

Élément	RAGbot	SciSpace Copilot
Type	Backend FastAPI personnalisé avec LangChain	Application SaaS propriétaire
Stockage	MongoDB + stockage vectoriel local (ChromaDB)	Stockage dans le cloud propriétaire
Traitement local	Oui (traitement en local avec FastAPI + Ollama)	Non (cloud uniquement)
Framework IA	LangChain, Ollama, ChromaDB	LangChain, FAISS, GPT-4
Langage supporté	Multilingue (anglais + français avec détection automatique)	Principalement anglais

Tableau 3 : Architecture Générale du système

SciSpace Copilot et SciSpace CloudBot ont tous deux pour objectif de permettre une interaction intelligente avec les documents textuels, mais leurs architectures varient considérablement en termes de mise en œuvre et de déploiement. SciSpace Copilot fonctionne sur une infrastructure SaaS centralisée, avec tout le traitement effectué sur les serveurs de l'éditeur, ce qui signifie que les utilisateurs n'ont aucun contrôle sur l'environnement d'exécution ou le traitement des données. Cela procure une expérience utilisateur fluide, mais cela pourrait soulever des inquiétudes quant à la confidentialité des documents. D'autre part, SciSpace CloudBot adopte une architecture décentralisée et personnalisée construite sur FastAPI, intégrant l'extraction de texte, la vectorisation, l'indexation, le stockage MongoDB ainsi que des modèles linguistiques locaux grâce à Ollama (LLaMA 3). Ce design permet un déploiement local ou privé, garantissant un contrôle total des données et une adaptabilité maximale à d'autres contextes linguistiques.

2. Fonctionnement RAG

Étape du pipeline	RAGbot	SciSpace
Extraction de texte	.pdf, .docx, .txt via UnstructuredPDFLoader	.pdf uniquement (scientifiques) avec OCR
Vectorisation (embeddings)	OllamaEmbeddings (multilingual- e5-base)	OpenAI embeddings
Indexation	ChromaDB par utilisateur	FAISS propriétaire
Modèle LLM	llama3.2:1b local via Ollama	GPT-3.5/4
Récupération	Recherche vectorielle (top-1 context)	Recherche vectorielle avancée (multi-passage)
Génération	Réponses multilingues (selon détection de langue)	Anglais uniquement

Tableau 4: fonctionnement RAG

La génération augmentée par la récupération (RAG) est au cœur de SciSpace Copilot et SciSpace. Elle combine l'extraction de documents pertinents avec la génération de texte à l'aide d'un modèle linguistique pour produire des réponses précises et contextualisées. SciSpace Copilot adopte une structure classique : les utilisateurs soumettent un article scientifique en format PDF, celui-ci est ensuite segmenté,

vectorisé à l'aide des embeddings d'OpenAI, puis indexé dans une base FAISS. Néanmoins, ce processus est optimisé pour les articles scientifiques et manque de souplesse. En revanche, le RAG adopte les mêmes principes, mais dans une mise en œuvre ouverte et personnalisable. Les utilisateurs ont la possibilité de sélectionner divers formats de documents, d'extraire du texte et de produire des réponses à l'aide d'un modèle linguistique local (LLaMA 3 via ChatOllama).

3. Comparaison visuelle

On a interrogé les deux systèmes la même question «quels sont les types d'apprentissage automatique» concernant le même fichier
Voici le contenu du fichier :

Le Machine Learning : Une Introduction

Le **machine learning** (apprentissage automatique) est une branche de l'intelligence artificielle (IA) qui permet aux ordinateurs d'apprendre à partir de données sans être explicitement programmés. Contrairement à la programmation traditionnelle, où les règles sont définies par le programmeur, dans le machine learning, l'algorithme apprend des données d'entrée et génère des prédictions ou des décisions basées sur ces données.

Il existe plusieurs types d'apprentissage en machine learning, notamment :

1. **L'apprentissage supervisé** : Dans ce type d'apprentissage, l'algorithme apprend à partir d'un ensemble de données étiquetées, c'est-à-dire que chaque exemple de données est accompagné de la réponse correcte. L'objectif est de prédire la sortie pour de nouvelles données non étiquetées. Par exemple, la classification des emails comme spam ou non-spam est un problème d'apprentissage supervisé.

2. **L'apprentissage non supervisé** : Contrairement à l'apprentissage supervisé, l'apprentissage non supervisé n'a pas de données étiquetées. L'algorithme doit trouver des motifs ou des structures cachées dans les données. Un exemple classique d'apprentissage non supervisé est la **segmentation de clients** basée sur leurs comportements d'achat.

3. **L'apprentissage par renforcement** : Ce type d'apprentissage consiste à entraîner un agent (un programme ou un robot) à prendre des décisions dans un environnement pour maximiser une récompense cumulative. Par exemple, un agent peut apprendre à jouer à un jeu vidéo en recevant des récompenses pour les actions qui mènent à une victoire.

Et voici les résultats affichés :

● Réponse de RAGbot

 Assistant 15:51:17

Les types d'apprentissage en machine learning sont les suivants :

1. ****Apprentissage Supervisé**** : L'algorithme apprend à partir d'un ensemble de données étiquetées et cherche à prédire la sortie pour de nouvelles données non étiquetées.
2. ****Apprentissage Non-Supervisé**** : L'algorithme cherche des motifs ou des structures cachés dans les données sans priorité au prédiction des résultats, par exemple la segmentation de clients sur leurs comportements d'achat.
3. ****Apprentissage Par Renforcement**** : L'algorithme prend des décisions dans un environnement pour maximiser une récompense cumulative et apprend à partir des expériences.

 Masquer le contexte ^

 Extrait le plus pertinent

Pertinence: 100%

 Fichier : test.pdf
Le **Machine Learning** : Une Introduction

Le **machine learning** (apprentissage automatique) est une branche de l'intelligence artificielle (IA) qui permet aux ordinateurs d'apprendre

● Réponse de Scispace

The screenshot shows a Scispace interface. The main document is titled "Le Machine Learning : Une Introduction". It defines machine learning as a branch of artificial intelligence (IA) that allows computers to learn from data without explicit programming. It mentions that rules are defined by the programmer, and the algorithm learns from input data to generate predictions or decisions based on that data. It also states that there are several types of machine learning, notably:

- 1. L'apprentissage supervisé :** Dans ce type d'apprentissage, l'algorithme apprend à partir d'un ensemble de données étiquetées, c'est-à-dire que chaque exemple de données est accompagné de la réponse correcte. L'objectif est de prédire la sortie pour de nouvelles données non étiquetées. Par exemple, la classification des emails comme spam ou non-spam est un problème d'apprentissage supervisé.

The sidebar on the right contains a section titled "Types d'apprentissage dans le domaine de l'apprentissage automatique". It explains that machine learning includes different types of methods that allow computers to learn from data. It lists the main types of machine learning:

- **Apprentissage supervisé :**
 - Cette méthode consiste à entraîner un algorithme sur un ensemble de données étiquetées, où chaque entrée est associée à une étiquette de sortie correspondante. L'objectif est d'apprendre une fonction qui mappe les entrées aux sorties, permettant au modèle de faire des prédictions sur de nouvelles données.
- **Apprentissage non supervisé :**
 - En apprentissage non supervisé, l'algorithme fonctionne avec des données non étiquetées. Il vise à identifier des modèles ou des structures au sein des données sans aucune indication explicite sur ce qu'il faut prévoir. Des techniques telles que le clustering et la réduction de dimensionnalité sont couramment utilisées dans cette approche.
- **Apprentissage semi-supervisé :**
 - Cette approche combine des données étiquetées et non étiquetées pour la formation. Il est particulièrement utile lorsqu'il est difficile ou coûteux d'obtenir un ensemble de données entièrement étiqueté. Le modèle apprend à partir des exemples étiquetés tout en utilisant les données non étiquetées pour améliorer ses performances.
- **Apprentissage par renforcement :**

Below the list, there is a button "Generate summary of this paper, Results of the paper, Conclusions from the |" and a link "+13 more". At the bottom of the sidebar, there is a "Chat With:" section with a text input "Ask any question..." and a "High Quality" toggle switch.

Types d'apprentissage dans le domaine de l'apprentissage automatique

L'apprentissage automatique comprend différents types de méthodes d'apprentissage qui permettent aux ordinateurs d'apprendre à partir de données. Voici les principaux types d'apprentissage dans le domaine de l'apprentissage automatique :

- **Apprentissage supervisé :**
 - Cette méthode consiste à entraîner un algorithme sur un ensemble de données étiquetées, où chaque entrée est associée à une étiquette de sortie correspondante. L'objectif est d'apprendre une fonction qui mappe les entrées aux sorties, permettant au modèle de faire des prédictions sur de nouvelles données.
- **Apprentissage non supervisé :**
 - En apprentissage non supervisé, l'algorithme fonctionne avec des données non étiquetées. Il vise à identifier des modèles ou des structures au sein des données sans aucune indication explicite sur ce qu'il faut prévoir. Des techniques telles que le clustering et la réduction de dimensionnalité sont couramment utilisées dans cette approche.
- **Apprentissage semi-supervisé :**
 - Cette approche combine des données étiquetées et non étiquetées pour la formation. Il est particulièrement utile lorsqu'il est difficile ou coûteux d'obtenir un ensemble de données entièrement étiqueté. Le modèle apprend à partir des exemples étiquetés tout en utilisant les données non étiquetées pour améliorer ses performances.

- **Apprentissage par renforcement :**

- L'apprentissage par renforcement implique un agent qui apprend à prendre des décisions en interagissant avec un environnement. L'agent reçoit des commentaires sous forme de récompenses ou de pénalités en fonction de ses actions, ce qui l'amène à apprendre des stratégies optimales au fil du temps.

- **Apprentissage autosupervisé :**

- Il s'agit d'une forme d'apprentissage non supervisé dans laquelle le modèle génère ses propres signaux de supervision à partir des données d'entrée. Cela implique souvent de prédire des parties de l'entrée à partir d'autres parties, ce qui permet au modèle d'apprendre des représentations utiles sans avoir besoin de données étiquetées.

- **Apprentissage par transfert :**

- L'apprentissage par transfert s'appuie sur un modèle pré-entraîné pour une tâche et l'adapte à une tâche différente mais connexe. Cela est particulièrement utile lorsque les données disponibles pour la nouvelle tâche sont limitées, car les connaissances acquises lors de la tâche initiale sont utilisées.

Ces types d'apprentissage illustrent les diverses méthodologies de l'apprentissage automatique, chacune étant adaptée à différents types de données et à différents scénarios de résolution de problèmes .(*Le Machine Learning: Une Introduction*, n.d.)

Profondeur des explications

RAGbot:

Il fournit des définitions concises mais complètes qui résument fidèlement les points clés du document source. Les explications sont directes et ne contiennent pas d'information supplémentaire non présente dans le document.

Il présente aussi l'information de manière numérotée et concise, reflétant la structure du document original.

SciSpace:

SciSpace offre des explications plus détaillées, pour les types d'apprentissage automatique, ces détails supplémentaires ne sont pas présents dans le document source.

Conclusion

- Exactitude: RAGbot est plus fidèle au document source, ne présentant que les informations qui y figurent réellement.
- Inventions de SciSpace: SciSpace a ajouté deux types d'apprentissage (semi-supervisé et par transfert) qui n'apparaissent pas dans le document source, ainsi que des détails techniques supplémentaires non présents dans le document.
- Contexte: RAGbot respecte strictement le contexte du document, tandis que SciSpace semble avoir incorporé des connaissances extérieures au document fourni.

En résumé, RAGbot présente une extraction fidèle du contenu du document source, tandis que SciSpace a enrichi sa réponse avec des informations supplémentaires qui ne figurent pas dans le document original, créant ainsi une présentation plus complète mais moins fidèle au contenu source spécifique.

Conclusion générale

Pour faire court, ce projet se penche minutieusement sur la conception et la réalisation d'un chatbot intelligent ancré dans l'approche RAG (Retrieval-Augmented Generation), mêlant les fonctionnalités poussées des modèles de langage tels que LLaMA 3 à des outils comme LangChain, Ollama et FastAPI. En raison des contraintes inhérentes aux systèmes traditionnels de traitement du langage naturel, tels que les hallucinations et la dépendance à une base de données statique, l'incorporation d'un pipeline RAG offre la possibilité d'augmenter la précision et la pertinence des réponses par le biais d'une récupération contextuelle dynamique de documents. Le système créé assure une gestion intégrale des documents, l'authentification des utilisateurs, le suivi des conversations, et offre une interface web dynamique via Next.js, conçue pour améliorer l'expérience utilisateur.

Grâce à une architecture modulaire, à l'indexation vectorielle et à l'exploitation d'une base documentaire personnalisée, ce projet offre une solution robuste et extensible aux besoins croissants en matière d'IA conversationnelle explicable, fiable et adaptée à des contextes métier spécifiques.

Références et bibliographies

- **Lewis, P., Oguz, B., Rinott, R., Riedel, S., & Stoyanov, V.** (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In Advances in Neural Information Processing Systems (NeurIPS).
<https://arxiv.org/abs/2005.11401>
- **Touvron, H., Lavril, T., Izacard, G., et al.** (2023). *LLaMA: Open and Efficient Foundation Language Models*. Meta AI.
<https://arxiv.org/abs/2302.13971>
- **LangChain Documentation.** (2024). *LangChain: Framework for Developing Applications Powered by Language Models*.
<https://docs.langchain.com>
- **Ollama Documentation.** (2024). *Run Large Language Models Locally*. Ollama.
<https://ollama.com>
- **Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K.** (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In NAACL.
<https://arxiv.org/abs/1810.04805>
- **OpenAI.** (2023). *GPT-4 Technical Report*. OpenAI.
<https://openai.com/research/gpt-4>
- **Zhang, Z., Zhao, H., & Chen, L.** (2023). *ChatGPT for Conversational Information Retrieval: A Review*. arXiv preprint.
<https://arxiv.org/abs/2303.04788>
- **Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-T.** (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. In EMNLP.
<https://arxiv.org/abs/2004.04906>
- **Mialon, G., Wang, J., & et al.** (2023). *Augmented Language Models: a Survey*. arXiv preprint.
<https://arxiv.org/abs/2302.07842>
- **ChromaDB Documentation.** (2024). *The Open-Source Embedding Database*.
<https://docs.trychroma.com>